AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ FIZYKI I INFORMATYKI STOSOWANEJ

KATEDRA INFORMATYKI STOSOWANEJ I FIZYKI KOMPUTEROWEJ

Projekt dyplomowy

Projekt i implementacja systemu wspomagającego planowanie harmonogramów pracy w branży gastronomicznej

Design and Implementation of a Work Schedule Management System for the Gastronomy Industry

Autor: Michał Zychowicz
Kierunek studiów: Informatyka Stosowana
Opiekun pracy: dr inż. Ewa Olejarz-Mieszaniec

Kraków, 2026

Dla Dziadka Zbyszka (1952–2026)
który mówił o mnie dobrze innym,
zanim nauczyłem się mówić dobrze o sobie.

"Regret is my constant companion.
Do not let it become yours."
— Albus Dumbledore, Fantastyczne zwierzęta:
Zbrodnie Grindelwalda (2018)

Spis treści

Wstęp **5**
Cel i zakres pracy 5
Motywacja wyboru tematu i znaczenie praktyczne 6

1. Analiza problemu **7**
1.1. Planowanie harmonogramów pracy 7
1.2. Specyfika branży gastronomicznej 7
1.3. Porównanie z uniwersalnym planowaniem harmonogramów 8
1.4. Przegląd istniejących rozwiązań 8

2. Model matematyczny problemu **9**
2.1. Definicja zmiennych decyzyjnych 9
2.2. Zestaw podstawowych ograniczeń 9
2.3. Funkcja celu i przykłady możliwych kryteriów optymalizacji 10
2.4. Zakres modelu i przyjęte założenia 11

3. Technologie i narzędzia **12**
3.1. Wybór platformy i języków programowania 12
3.2. OR-Tools – biblioteka solvera CP-SAT 12
3.3. Architektura systemu 13

4. Projekt systemu **15**
4.1. Wymagania funkcjonalne 15
4.2. Wymagania niefunkcjonalne 22
4.3. Projekt bazy danych 22
4.3.1. Główne encje systemu 22
4.3.2. Relacje pomiędzy encjami 23
4.3.3. Uzasadnienie przyjętego modelu danych 24
4.3.4. Powiązanie bazy danych z wymaganiami funkcjonalnymi 25
4.3.5. Schemat ERD 26
4.4. Projekt API i podział modułów 27

5. Implementacja **28**
5.1. Backend i baza danych 28
5.2. Integracja z solverem OR-Tools 29
5.2.1. Model decyzyjny i ograniczenia twarde 29
5.2.2. Funkcja celu i ograniczenia miękkie 31
5.3. Interfejs użytkownika 34
5.3.1. Opis głównych ekranów 35

6. Testy i ewaluacja **41**
6.1. Scenariusze testowe systemu 41
6.2. Testy funkcjonalne 44
6.2.1. Analiza poprawności przepływu danych 44
6.2.2. Stabilność działania 45
6.3. Analiza wydajności algorytmu planowania 45
6.4. Porównanie planowania ręcznego i automatycznego 45

7. Podsumowanie i wnioski **47**
7.1. Osiągnięte cele pracy 47
7.2. Ograniczenia rozwiązania 47
7.3. Możliwości dalszego rozwoju 48

Bibliografia **49**

Spis rysunków **50**

Spis tabel **51**

Spis kodów źródłowych **52**

Wstęp

Jednym z najbardziej dynamicznych, nieprzewidywalnych i zmiennych środowisk pracy jest branża gastronomiczna. Wahania natężenia ruchu klientów zależne od pogody, pory dnia, dnia tygodnia czy sezonu powodują, że efektywne zaplanowanie harmonogramu pracy jest niemal niemożliwe. W wielu przedsiębiorstwach to zadanie jest realizowane manualnie przy użyciu papierowych grafików, procesorów tekstu lub arkuszy kalkulacyjnych, co nierzadko prowadzi do błędów oraz utrudnia komunikację w zespole.

Efektywne tworzenie i aktualizowanie harmonogramów pracy w gastronomii wymaga uwzględnienia wielu czynników, takich jak dyspozycyjność poszczególnych pracowników, ich preferencje, zapotrzebowania kadrowe, regulacje prawne w Kodeksie Pracy czy specyfikacja lokalu. Logicznym kierunkiem rozwoju jest zatem zastosowanie narzędzi informatycznych, które pomogą osobom odpowiedzialnym za ten proces.

Cel i zakres pracy

Celem niniejszej pracy jest projekt i implementacja aplikacji wspomagającej planowanie harmonogramów pracy w branży gastronomicznej, uwzględniając zarówno potrzeby menadżera, jak i pracowników. System ma umożliwiać intuicyjne tworzenie i edytowanie grafików, definiowanie stanowisk oraz zarządzanie personelem, a także zapewnić pracownikom dostęp do informacji o przydzielonych zmianach.

Zakres pracy obejmuje analizę problemu organizacji czasu pracy w gastronomii, identyfikację wymagań funkcjonalnych i niefunkcjonalnych systemu, projekt architektury oraz implementację aplikacji webowej opartej na bazie danych `PostgreSQL`. Ponadto w ramach pracy zaimplementowano moduł automatycznego generowania harmonogramów z wykorzystaniem solvera `CP-SAT` z biblioteki `Google OR-Tools`. Moduł ten realizuje podstawowy model optymalizacyjny przydzielający pracowników do zmian na podstawie zadeklarowanej dostępności oraz kluczowych ograniczeń i parametrów.

W ramach pracy przeprowadzono również testy funkcjonalne weryfikujące poprawność działania aplikacji oraz oceniono skuteczność w kontekście jej zastosowania w środowisku gastronomicznym. Zakres nie obejmuje implementacji zaawansowanych modeli predykcyjnych dotyczących zapotrzebowania kadrowego ani integracji z zewnętrznymi systemami raportowo-kadrowymi, jednak projekt został przygotowany w sposób umożliwiający przyszłą rozbudowę w tych kierunkach.

Zaprojektowany system łączy w sobie praktyczne potrzeby branży gastronomicznej z nowoczesnymi metodami optymalizacji procesów. W rezultacie może przyczynić się do oszczędności nakładu pracy, poprawy satysfakcji pracowników oraz ograniczenia liczby błędów lub niejasności w harmonogramach. Niniejsza praca prezentuje cały proces projektowy od analizy wamagań,

przez projekt architektury i implementację, aż po weryfikację działania i ocenę użyteczności opracowanego rozwiązania.

Motywacja wyboru tematu i znaczenie praktyczne

Planowanie harmonogramów pracy w branży gastronomicznej jest zadaniem szczególnie wymagającym, czasochłonnym i podatnym na błędy. Dynamiczny i małoprzewidywalny charakter pracy, częste nieobecności i zmienna dyspozycyjność personelu spowodowana nauką w szkole, na studiach lub nierzadko również łączeniem prac w wielu miejscach powodują, że tradycyjne metody organizacji grafiku takie jak komunikacja ustna, wiadomości wysyłane w różnych formatach, papierowe grafiki, procesory tekstu czy arkusze kalkulacyjne okazują się niewystarczające.

Wybór tematu dodatkowo był motywowany praktycznymi doświadczeniami z działania lokali gastronomicznych. Często gromadzenie danych o dyspozycyjności odbywa się w sposób rozproszony, gdzie część osób przesyła ją w wiadomościach prywatnych, inni przy pomocy osobnych plików, a niektórzy zapominają zrobić to w terminie. Wynikiem są niekompletne dane i zwiększone ryzyko pomyłek. Dodatkowo zdarzają się sytuacje, gdzie błędnie odczytane wiersze w arkuszu lub niewłaściwie oznaczone daty skutkują nieobecnośnią pracownika lub błędnie przypisanymi zmianami.

Proces układania harmonogramu pracy wymaga również zachowania sprawiedliwości i równomiernego rozłożenia zmian w skali miesiąca, co w przypadku pracy manualnej oznacza wielokrotne sprawdzanie dyspozycyjności, szacowanie liczby godzin i rotację pracowników. W połączeniu z presją czasu to zadanie staje się nie tylko czasochłonne, ale przy dużej liczbie pracowników również stresujące i bardziej podatne na błędy ludzkie.

Projekt aplikacji pozwala rozwiązać te problemy poprzez centralizację danych, ujednolicenie formy zgłaszania dyspozycyjności oraz automatyzację części procesu przydzielania zmian. Dodatkowo zastosowanie algorytmu optymalizacyjnego opartego na solverze `CP-SAT` umożliwia generowanie grafików uwzględniających dyspozycyjność pracowników czy ograniczenia czasowe.

Znaczenie praktyczne projektu wynika z realnych i wielokrotnie zauważalnych problemów w gastronomii. Zaproponowane rozwiązanie ma potencjał, aby zwiększyć efektywność operacyjną oraz ograniczyć liczbę konfliktów i błędów wynikających z pracy manualnej.

1. Analiza problemu

1.1. Planowanie harmonogramów pracy

Planowanie harmonogramów pracy jest jednym z najbardziej znaczących elementów zarządzania zasobami ludzkimi w branżach usługowych. Zadanie to polega na przypisywaniu członków personelu do konkretnych zmian na konkretne stanowiska w określonym przedziale czasowym. Należy jednak pamiętać o wielu ograniczeniach, które ten proces komplikują. Są to między innymi wymagania prawne, zapotrzebowanie operacyjne przedsiębiorstwa, kompetencje i uprawnienia pracowników oraz ich dostępność.

Harmonogram pracy powinien zapewniać ciągłość działania przedsiębiorstwa z zachowaniem efektywności ekonomicznej oraz komfortu pracy personelu. W praktyce oznacza to balansowanie między interesem pracodawcy a oczekiwaniami pracowników, co czyni proces zadaniem szczególnie wymagającym i podatnym na błędy.

Tradycyjne metody układania grafików, najczęściej oparte na arkuszach kalkulacyjnych, są czasochłonne oraz poziom ich skomplikowania rośnie liniowo wraz z liczbą pracowników, zmian oraz stanowisk. Brak automatyzacji prowadzi często do konfliktów harmonogramu, nadgodzin czy nierównomiernego obciążenia pracą.

1.2. Specyfika branży gastronomicznej

Według badań przytaczanych przez Stanisława Lipca wykonanych w latach 2021-2022 polska branża gastronomiczna liczyła wtedy ponad 500 tysięcy osób [1, s. 19], w której większość stanowili studenci oraz ludzie młodzi. Warto zaznaczyć, iż tę liczbę bardzo trudno oszacować z powodu braku danych statystycznych oraz zależy ona na przykład od sytuacji kraju. ”Przykładowo, z powodu epidemii koronawirusa w 2020 i 2021 roku z branży odeszło około 200 tysięcy pracowników” . Branża charakteryzuje się wysoką zmiennością zapotrzebowania na pracę oraz efektywności operacyjnej. Natężenie ruchu sprzedaży zależy od wielu czynników, takich jak pogoda, pora dnia, dzień tygodnia, sezonowość czy wydarzenia specjalne. Powoduje to wymóg elastycznego dostosowywania liczby pracowników na zmianach.

Dodatkowym wyzwaniem jest rotacja personelu oraz zatrudnianie pracowników dorywczych, o którym wspomina Stanisław Lipiec:

> W barach i restauracjach ponad połowa pracujących to studenci, pozostali to pracownicy tymczasowi pracujący zazwyczaj krócej niż przez rok. [...] wolą wszystkich stron [pracowników i pracodawców - przypis autora] jest wiązanie się tymczasowymi, dynamicznymi, elastycznymi i tanimi kontraktami prawa cywilnego lub zupełne działanie w szarej strefie. [1, s. 42]

Powyższe zjawiska sprawiają, że kluczowym czynnikiem staje się dyspozycyjność oraz elastyczność pracowników, a także ich różnice kompetecyjne, ponieważ nie każdy pracownik może

pracować na każdym stanowisku. Dodatkowo, w celu zachowania zdrowych relacji między życiem zawodowym a prywatnym, pracodawcy, w miarę możliwości, powinni uwzględniać preferencje pracowników dotyczące własnych zmian, takie jak wybór zmian czy ograniczenia dyspozycyjności.

1.3. Porównanie z uniwersalnym planowaniem harmonogramów

Uniwersalne systemy planowania harmonogramów znajdują zastosowanie w wielu branżach, takich jak administracja, produkcja czy logistyka. Zazwyczaj godziny pracy są stałe, powtarzają się w cyklach oraz pracownicy mają jasno określone swoje obowiązki. Dzięki temu proces planowania może być zautomatyzowany.

W przeciwieństwie do takich rozwiązań, sektor gastronomiczny wymaga indywidualnego podejścia. Zmiany robocze mają różne długości, a zapotrzebowanie na personel gwałtownie się zmienia. Dodatkowo pracownicy deklarują zmienną dostępność, przy czym w części przypadków dyspozycyjność obejmuje jedynie kilka dni w miesiącu.

Uniwersalne systemy planowania nie zawsze pozwalają na tak złożone i zmienne podejście, co wymaga konieczności ręcznych modyfikacji grafiku lub stosowania obejść, które obniżają czytelność i spójność harmonogramu pracy. W efekcie proces staje się skomplikowany, maleje efektywność planowania, a wzrasta ryzyko popełniania błędów. Zjawiska te uzasadniają potrzebę stosowania rozwiązań dedykowanych branży gastronomicznej, które uwzględniają jej specyfikę oraz pozwalają na bardziej precyzyjne i elastyczne planowanie harmonogramów pracy.

1.4. Przegląd istniejących rozwiązań

Rozwiązań wspierających planowanie harmonogramów pracy dostępnych na rynku jest wiele. Można wyróżnić grupę narzędzi dedykowaną branży gastronomicznej, jak i narzędzia ogólne, przeznaczone do ewidencji czasu pracy oraz zarządzania kadrami.

Jednym z systemów wykorzystywanych w polskiej gastronomii jest program `izzyRest`. Jego głównym celem jest obsługa sprzedaży, zarządzanie zamówieniami oraz raportowanie i analiza danych operacyjnych. W kontekście planowania pracy `izzyRest` oferuje uniwersalne i podstawowe mechanizmy, które są niewystarczająco elastyczne.

Drugim przykładem rozwiązania jest `Kadromierz`, czyli program skoncentrowany na ewidencji czasu pracy oraz obsłudze procesów kadrowych. Narzędzie to umożliwia rejestrowanie godzin pracy, nieobecności, urlopów oraz generowanie zestawień i raportów kadrowych. `Kadromierz` precyzyjnie rozlicza czas pracy i sprawdza zgodność zmian z przepisami prawa pracy. Jednak jako narzędzie ogólne nie uwzględnia specyfiki branży gastronomicznej.

Analiza powyższych rozwiązań wskazuje, że rozwiązania dostępne na rynku koncentrują się albo na szeroko pojętym zarządzaniu lokalem gastronomicznych, albo na aspektach kadrowych. Brakuje narzędzia, które uwzględnia dynamiczny charakter pracy w gastronomii oraz realne potrzeby użytkowników.

2. Model matematyczny problemu

2.1. Definicja zmiennych decyzyjnych

Aby formalnie opisać problem planowania harmonogramów pracy trzeba zdefiniować zmienne decyzyjne, które wpływają na decyzje modelu matematycznego [2].

W rozważanym problemie najbardziej znaczącą decyzją jest przypisanie konkretnego pracownika do konkretnej zmiany roboczej w danym dniu. Oznacza to, że podstawową zmienną decyzyjną jest zmienna binarna informująca czy dany pracownik jest przypisany do określonej zmiany. Zmienna ta zależy od trzech indeksów: pracownika, dnia oraz zmiany roboczej.

Zmienna decyzyjna przyjmuje wartość równą 1, gdy pracownik zostaje przypisany do zmiany, oraz w przeciwnym wypadku wartość równą 0. Binarny typ zmiennej skutkuje prostą interpretację wyników [3].

Zdefiniowane zmienne binarne stanowią podstawę do dalszego formułowania ograniczeń modelu oraz funkcji celu. Na ich podstawie możliwe jest uwzględnienie dostępności pracowników, obsady zmian, a także kryteriów optymalizacji związanych z jakością wygenerowanego harmonogramu pracy. Zestawienie zmiennych decyzyjnych zastosowanych w modelu przedstawiono w tabeli 2.1.

Tabela 2.1: Zestawienie zmiennych decyzyjnych modelu

Symbol	**Opis**	**Zakres wartości**
$x_{i,d,s}$	Przypisanie pracownika i do zmiany s w dniu d	$x_{i,d,s} \in \{0, 1\}$

Źródło: opracowanie własne

2.2. Zestaw podstawowych ograniczeń

Aby generowany harmonogram pracy spełniał rzeczywiste warunki funkcjonowania lokalu gastronomicznego oraz był wykonalny, potrzebne są ograniczenia. Reprezentowane są przez wyrażenia matematyczne, których celem jest ograniczanie wartości lub relacji między nimi [2]. Ograniczenia modelu obejmują w szczególności:

— brak nakładania się zmian przypisanych do jednego pracownika,
— pozostawienie zmiany jako otwartej w przypadku braku dostępnych pracowników,
— zgodność przypisań z zadeklarowaną dyspozycyjnością pracowników,
— uwzględnienie kompetencji wymaganych do realizacji danej zmiany,
— dostosowanie liczby przypisanych zmian do wymiaru czasu pracy pracownika.

Model zachowuje elastyczność planowania w sytuacjach wyjątkowych. W najgorszym przypadku, jeżeli dana zmiana nie będzie mogła zostać obsadzona, pozostaje ona nieprzypisana i może zostać uzupełniona ręcznie lub poprzez dobrowolny zapis pracownika spełniającego wymagania danej zmiany.

W modelu możliwe jest uwzględnienie różnych form zatrudnienia oraz indywidualne priorytety przypisań. W przypadku pracowników pracujących na umowie o pracę zakłada się konieczność zrealizowania określonej ilości godzin w danym okresie rozliczeniowym. Dla pracowników nieetatowych model umożliwia ustalenie preferencji przypisań w sposób elastyczny poprzez wagi ustalające docelową liczbę godzin pracy. Pozwala to wyróżnić pracowników preferowanych, którzy otrzymują dodatkowe zmiany, pracowników neutralnych, dla których zmiany są rozdzielane możliwie po równo, oraz pracowników traktowanych jako rozwiązanie „ostateczne".

Dodatkowym aspektem uwzględnionym w modelu jest rozkład zmian w czasie. Prócz łącznej liczby godzin/zmian istotne jest również, aby przypisania nie były skoncentrowane w krótkim okresie, lecz możliwie jak najbardziej równomiernie. Zbyt duże skupienie zmian może prowadzić bowiem do obniżenia jakości pracy. Kryterium to wpływa na ocenę jakości grafiku poprzez funkcję celu.

Ze względu na charakter problemu wyróżnia się dwa rodzaje ograniczeń: twarde oraz dodatkowe (miękkie). Przykładowy podział ograniczeń zastosowanych w modelu przedstawiono w tabeli 2.2. Ograniczenia twarde muszą być spełnione bezwzględnie, natomiast ograniczenia dodatkowe wpływają na jakość rozwiązania i są uwzględniane w funkcji celu modelu optymalizacyjnego [4].

Tabela 2.2: Podział ograniczeń w modelu planowania grafiku pracy

Ograniczenia twarde	**Ograniczenia dodatkowe**
Brak nakładania się zmian przypisanych do jednego pracownika	Preferowane proporcje godzin pracy dla pracowników nieetatowych
Zgodność przypisań z deklarowaną dostępnością czasową	Równomierny rozkład zmian w czasie (unikanie skupienia zmian)
Spełnienie wymaganej liczby godzin przez pracowników etatowych	Równomierny podział pozostałych zmian pomiędzy pracowników neutralnych
Wymagane kompetencje do realizacji danej zmiany	Minimalizacja przypisań realizowanych w ostateczności

Źródło: opracowanie własne

W przedstawionym modelu nie uwzględniono bezpośrednich ograniczeń wynikających z przepisów prawa pracy, takich jak maksymalny tygodniowy czas pracy czy minimalny okres odpoczynku. Ograniczenia te mogą zostać dodane w przyszłości jako dodatkowe ograniczenia twarde, w celu dostosowania aplikacji do obowiązujących regulacji prawnych [5].

2.3. Funkcja celu i przykłady możliwych kryteriów optymalizacji

Funkcja celu umożliwia ocenę jakości wygenerowanego harmonogramu pracy poprzez przypisanie wartości (ocenianie) każdego rozwiązania. Jej zadanie to wybór najlepszego rozwiązania

spośród wszystkich, które spełniają ograniczenia twarde, z uwzględnieniem ograniczeń dodatkowych [3, 4].

W rozważanym problemie optymalny harmonogram pracy to taki, który spełnia wszystkie ograniczenia twarde wymienione w tabeli 2.2. Dopiero wtedy funkcja celu służy do porównywania rozwiązań na podstawie ograniczeń dodatkowych, które wpływają na komfort pracy oraz równowagę obciążenia personelu.

Na podstawie przedstawionych założeń funkcja celu może uwzględniać następujące kryteria optymalizacji:

— spełnienie wymaganej liczby godzin przez pracowników etatowych,
— przydział zmian pracownikom nieetatowym zgodnie z ustalonymi udziałami procentowymi,
— możliwie równomierny rozkład zmian w czasie w obrębie całego okresu planowania,
— równomierny podział pozostałych zmian pomiędzy pracowników neutralnych,
— minimalizacja liczby przypisań realizowanych w sytuacjach ostatecznych.

Poszczególne kryteria mogą być łączone w jeden spójny mechanizm oceny jakości harmonogramu. Takie podejście pozwala na elastyczne dostosowanie planowania do rzeczywistych potrzeb lokalu gastronomicznego.

2.4. Zakres modelu i przyjęte założenia

Zaproponowany model planowania harmonogramu pracy koncentruje się na statycznym problemie przydziału pracowników do konkretnych zmian w danym okresie czasu. Model zakłada, iż wszystkie dane wejściowe, takie jak dyspozycyjność pracowników, preferencje przypisań oraz zapotrzebowanie personelu, są znane przed rozpoczęciem procesu optymalizacji i nie ulegają zmianie w trakcie.

Model nie przewiduje dynamicznych wydarzeń, takich jak nagła nieobecność pracownika. Dodatkowo ograniczenia wynikające z prawa pracy również nie zostały bezpośrednio zaimplementowane, jednak struktura modelu umożliwia dodanie ich w przyszłości.

3. Technologie i narzędzia

3.1. Wybór platformy i języków programowania

Projektowany system został zrealizowany jako aplikacja webowa, która działa w architekturze klient - serwer. Wybór takiej platformy wynika z zapewnienia łatwego dostępu do systemu z poziomu różnych urządzeń, bez konieczności instalowania dodatkowych programów po stronie klienta.

Część serwerowa systemu została zrealizowana przy użyciu języka PHP, który umożliwia efektywne przetwarzanie danych oraz komunikację z bazą danych. Wybór tego języka uzasadniony jest wsparciem w środowiskach hostingowych oraz dużą liczbą bibliotek.

Do realizacji warstwy prezentacji wykorzystano języki HTML oraz CSS, odpowiedzialne za strukturę strony i wygląd interfejsu. Pozwalają one na stworzenie nowoczesnego i responsywnego interfejsu, dostosowanego do potrzeb użytkowników. Dodatkowo zastosowano język JavaScript w celu obsługi dynamicznych elementów strony.

Dane systemu są przechowywane w relacyjnej bazie danych, obsługiwanej przy pomocy języka PostreSQL. Takie rozwiązanie umożliwia przechowywanie danych w ustrukturyzowanej formie oraz realizację skomplikowanych zapytań.

3.2. OR-Tools – biblioteka solvera CP-SAT

Do rozwiązania problemu generowania harmonogramu wykorzystano bibliotekę OR-Tools, stworzoną przez firmę Google. Jest to narzędzie open source służące do modelowania i rozwiązywania problemów optymalizacyjnych [6].

Jednym z komponentów biblioteki OR-Tools jest solver CP-SAT (ang. *Constraint Programming - Satisfiability*), który łączy optymalizację ograniczeń z metodami rozwiązywania problemów spełnialności formuł logicznych [7, 8].

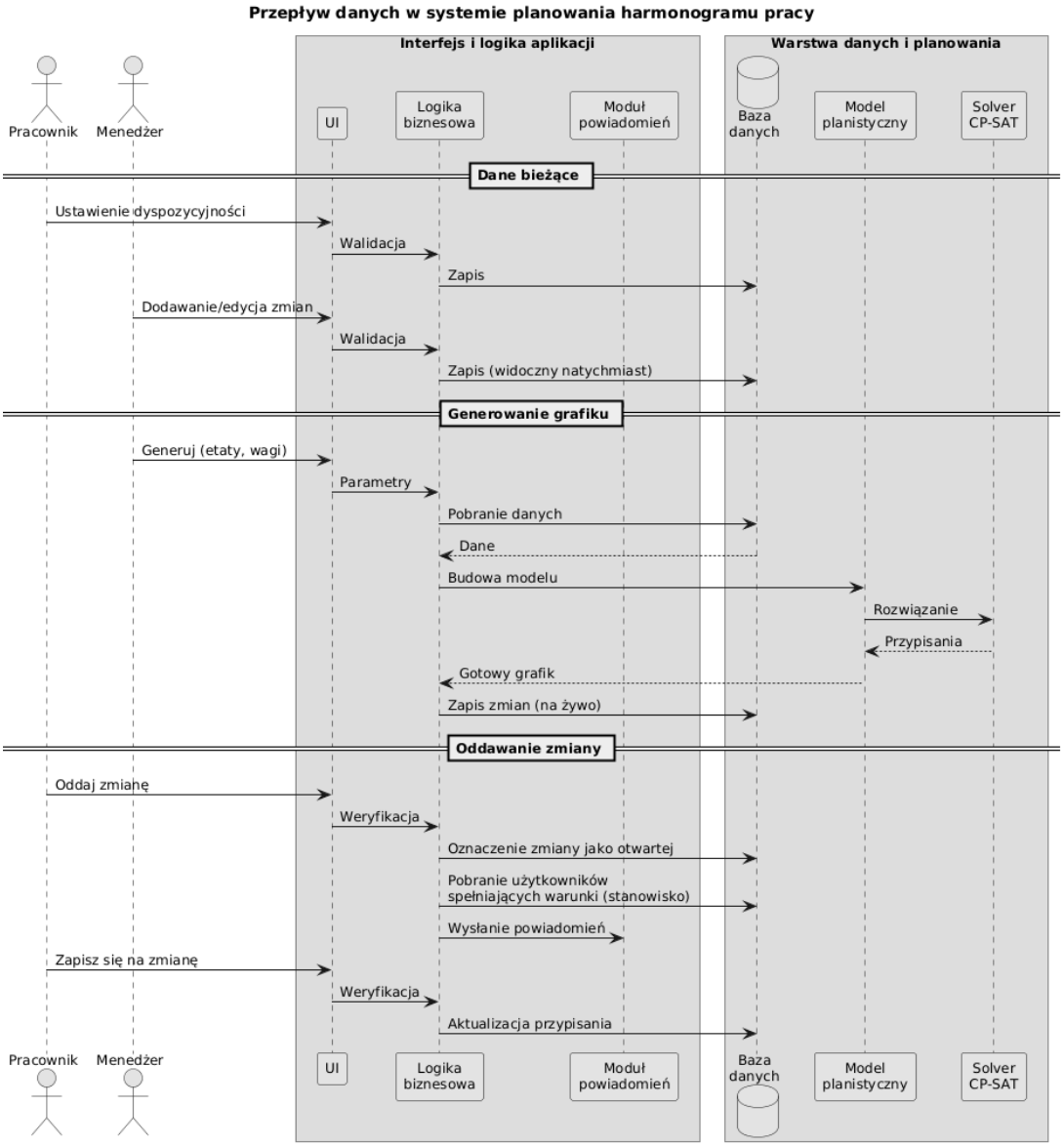
Solver został wybrany przez charakter problemu oraz przez jego wskazywanie w literaturze jako narzędzia odpowiedniego do planowania harmonogramów dla pracowników, które jest problemem kombinatorycznym [7]. W ramach niniejszego projektu położono większy nacisk na spełnianie ograniczeń modelu niż jego optymalizację.

W projekcie wykorzystano podstawowe możliwości biblioteki OR-Tools, takie jak:
— definiowanie zmiennych decyzyjnych typu binarnego i całkowitoliczbowego,
— modelowanie relacji logicznych oraz zależności czasowych pomiędzy zmiennymi,
— nakładanie ograniczeń zapewniających zgodność grafiku z dyspozycyjnością pracowników,
— wymuszanie braku konfliktów pomiędzy przypisanymi zmianami,
— obsługę sytuacji, w których zmiana pozostaje nieprzypisana do żadnego pracownika.

Zastosowanie biblioteki OR-Tools oraz solvera CP-SAT pozwoliło na stworzenie czytelnego i rozszerzalnego modelu, który może być w przyszłości rozwijany o dodatkowe ograniczenia lub kryteria decyzyjne.

3.3. Architektura systemu

Tworzony system posiada jednoznaczny podział odpowiedzialności pomiędzy poszczególne komponenty systemu oraz zapewnia przejrzysty przepływ danych. Na rysunku 3.1 przedstawiono przepływ danych w systemie.



Rysunek 3.1: Przepływ danych w systemie planowania harmonogramu pracy

Źródło: opracowanie własne

Architektura składa się z czterech podstawowych warstw: warstwy prezentacji, warstwy logiki biznesowej, warstwy dostępu do danych i warstwy bazy danych. Każda z nich realizuje określone zadania i za pomocą interfejsów komunikuje się z pozostałymi warstwami [9].

Przepływ danych rozpoczyna się na etapie ciągłego uzupełniania danych przez użytkowników. Pracownicy wprowadzają swoją dyspozycyjność, a menedżer definiuje zmiany robocze i stanowiska, które są zapisywane w warstwie bazy danych. Dane te są przechowywane w bazie danych i stanowią dane wejściowe wykorzystywane podczas generowania harmonogramu pracy.

Proces szukania rozwiązania jest inicjowany przez menedżera, gdzie wprowadzane są parametry planistyczne, takie jak etaty czy wagi liczby godzin. Parametry te nie są zapisywane wZbazie danych, tylko są przekazywane do warstwy logiki biznesowej również jako dane wejściowe.

Warstwa logiki biznesowej łączy dane wejściowe z bazy danych i wprowadzone przez menedżera oraz na tej podstawie budowany jest model, który po walidacji jest przekazywany do solvera CP-SAT biblioteki OR-Tools.

Po znalezieniu rozwiązania w postaci przypisań pracowników do konkretnych zmian, wynik jest interpretowany oraz zapisywany w bazie danych jako szkic wygenerowanego harmonogramu, gdzie menedżer może wprowadzić zmiany ręcznie, a następnie harmonogram jest udostępniany użytkownikom.

4. Projekt systemu

4.1. Wymagania funkcjonalne

Wymagania funkcjonalne opisują działania systemowe oraz metody komunikacji użytkowników z funkcjami aplikacji [10]. Zdefiniowane są dwie podstawowe role użytkowników: pracownik oraz menedżer. Pracownik jest odpowiedzialny za dostarczanie danych wejściowych w postaci swojej dyspozycyjności, a menedżer zarządza strukturą organizacyjną lokalu, zmianami roboczymi oraz ma możliwość generowania harmonogramu.

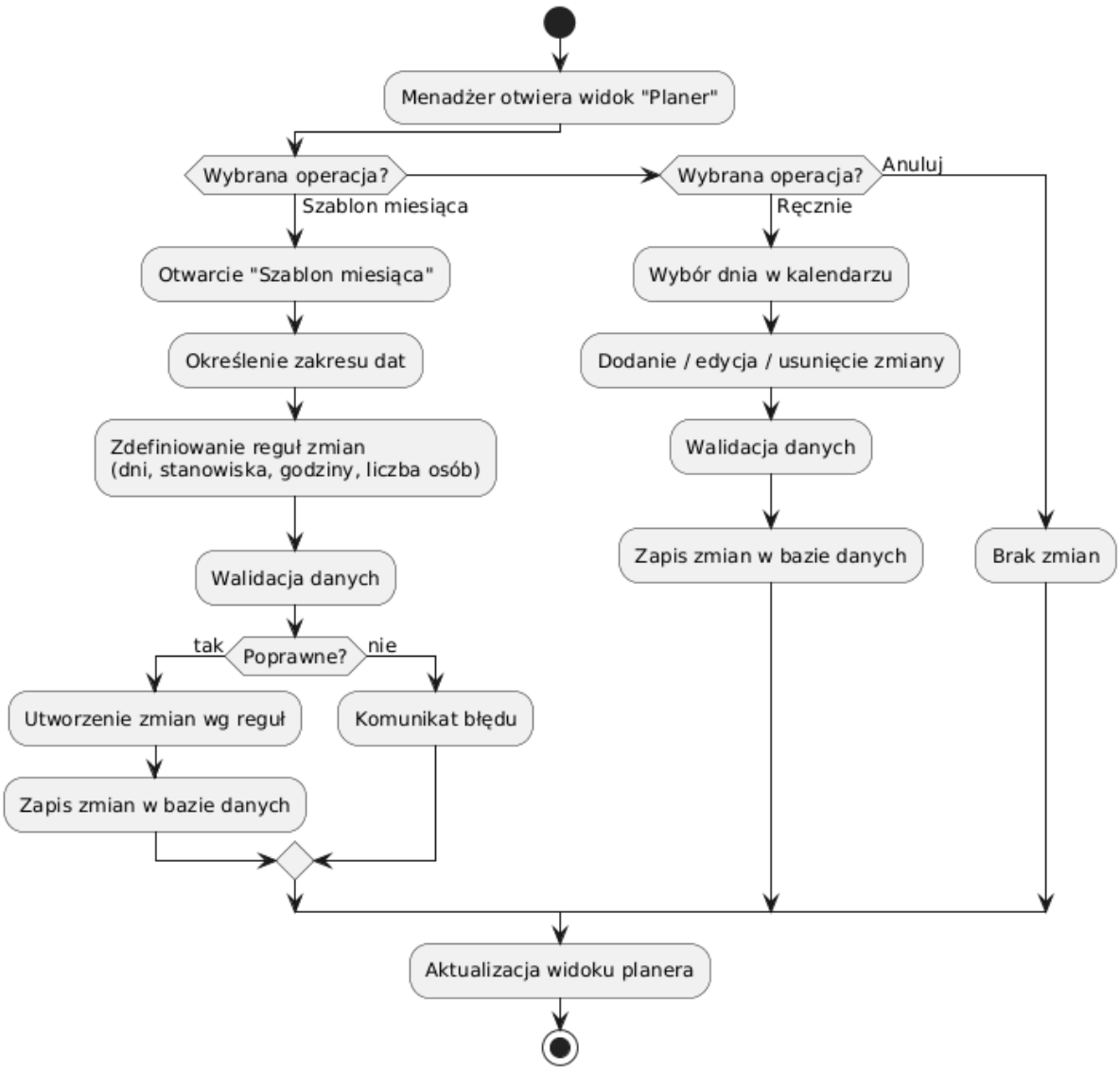
Do podstawowych wymagań funkcjonalnych systemu należą:

— obsługa użytkowników z podziałem na role: pracownik oraz menedżer,
— dodawanie, edycja oraz dezaktywacja pracowników,
— definiowanie stanowisk pracy oraz przypisywanie pracowników do stanowisk,
— wprowadzanie i modyfikowanie dyspozycyjności pracowników,
— definiowanie zmian roboczych, obejmujących czas trwania, liczbę wymaganych pracowników oraz stanowisko,
— określanie parametrów planistycznych, takich jak wymiar etatu lub wagowy udział czasu pracy,
— uruchamianie procesu generowania harmonogramu pracy,
— prezentacja wygenerowanego harmonogramu w formie graficznej,
— dopuszczenie pozostawienia zmiany jako nieobsadzonej w przypadku braku rozwiązania wykonalnego,
— zapis i odczyt danych wejściowych oraz wyników harmonogramu w bazie danych.

Rysunek 4.1: Diagram przypadków użycia systemu planowania harmonogramu pracy

Źródło: opracowanie własne

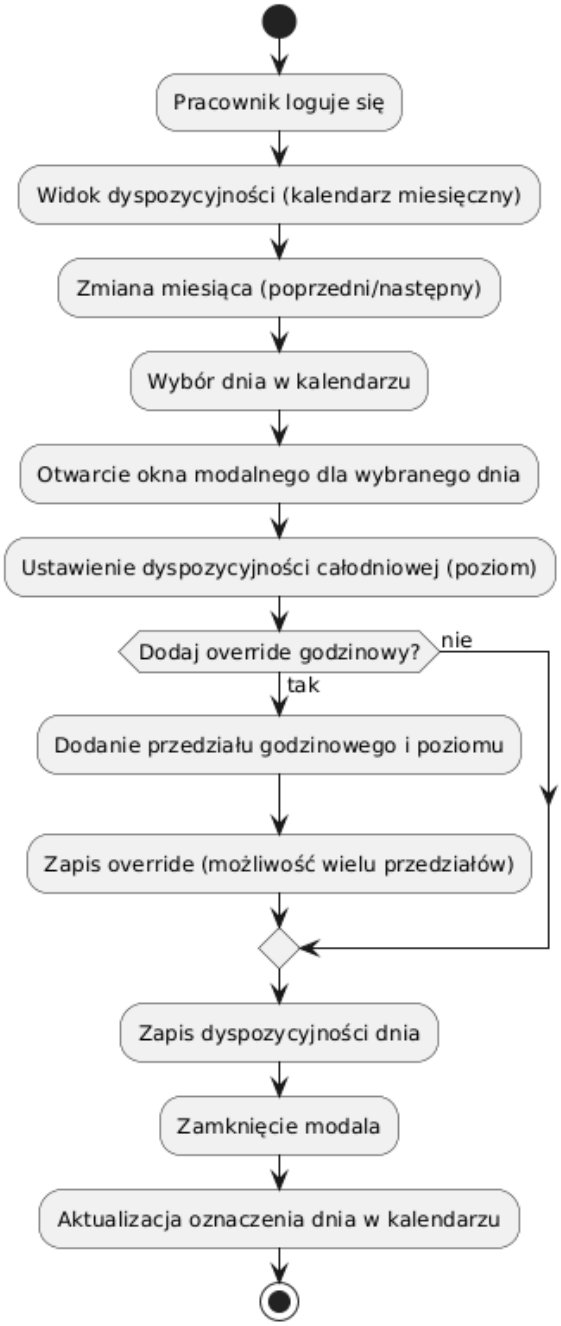
Diagram przypadków użycia przestawiony na rysunku 4.1 pokazuje funkcjonalności systemu z perspektywy użytkowników oraz różnice między rolami. Aktor menedżera przedstawiony został jako specjalizacja aktora pracownika, co odzwierciedla realną sytuację, gdy dany pracownik odpowiedzialny za planowanie grafiku również pracuje na stanowiskach.



Rysunek 4.2: Diagram aktywności procesu generowania harmonogramu pracy

Źródło: opracowanie własne

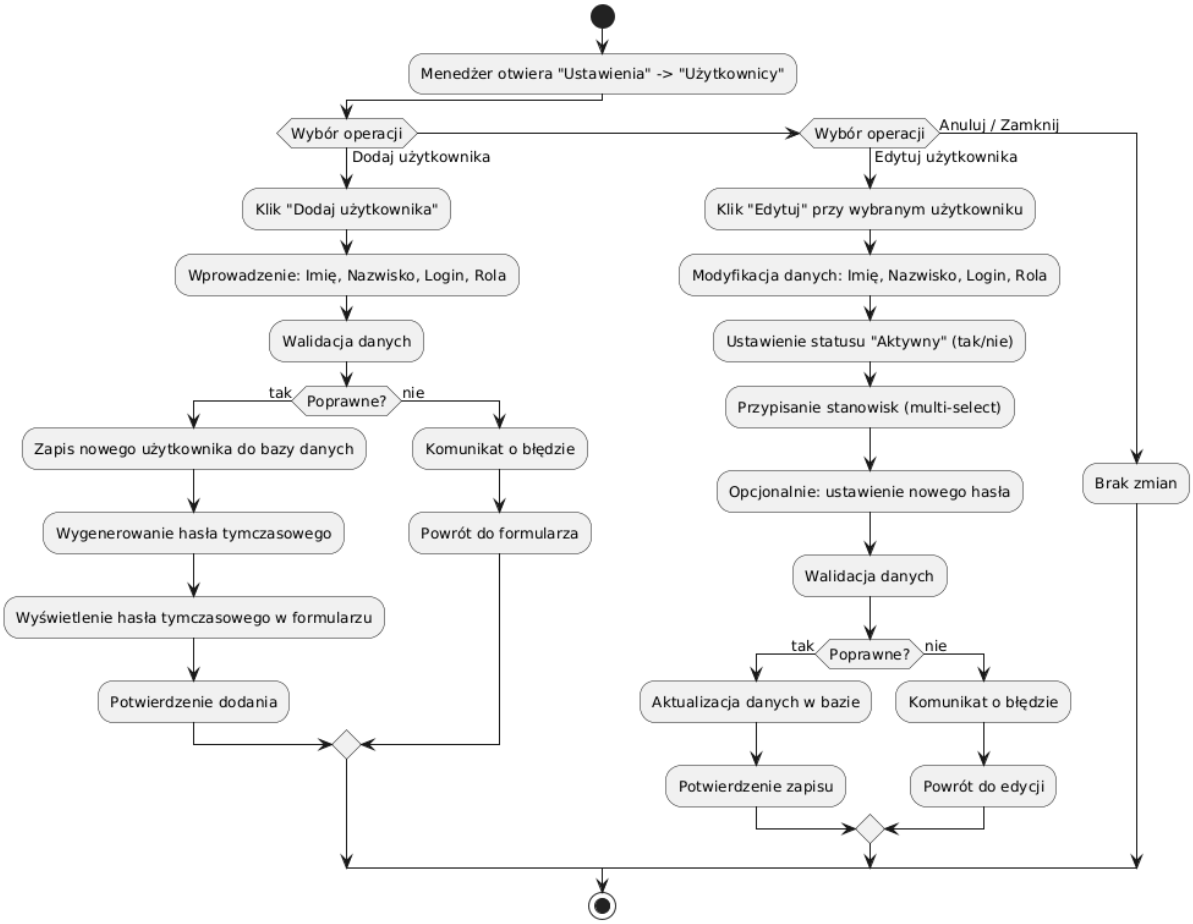
Diagram aktywności procesu generowania harmonogramu pracy przedstawiony na rysunku 4.2 pokazuje czynności realizowane przez menedżera i system. Diagram uwzględnia walidację danych wejściowych, budowy modelu oraz obsługę wyjątków, w których nie zostaje znalezione rozwiązanie.



Rysunek 4.3: Diagram aktywności procesu wprowadzania dyspozycyjności pracownika

Źródło: opracowanie własne

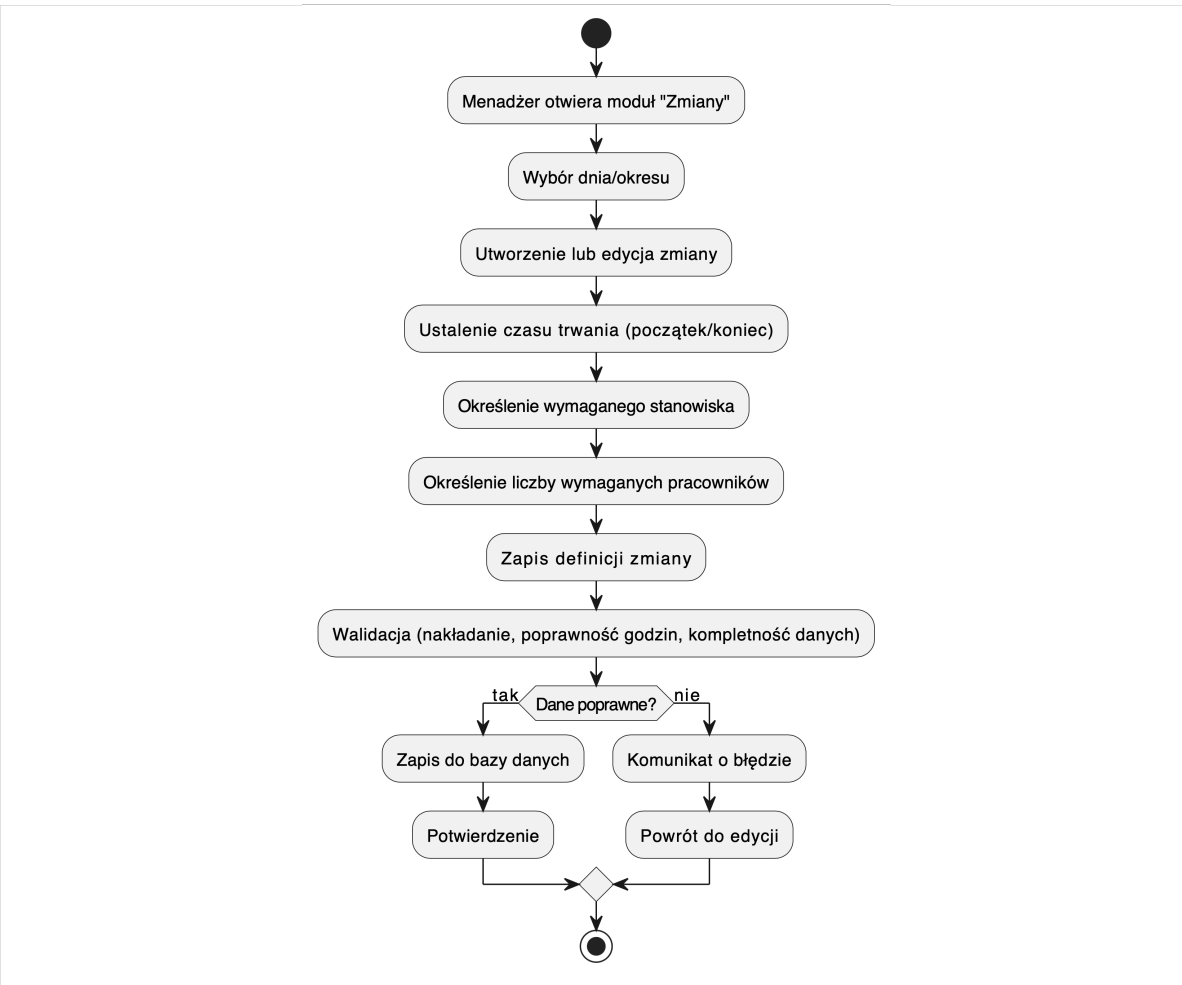
Diagram aktywności wprowadzania dyspozycyjności przedstawiony na rysunku 4.3 pokazuje proces przekazywania informacji o dostępności pracownika i zapisem do bazy danych.



Rysunek 4.4: Diagram aktywności procesu zarządzania pracownikami

Źródło: opracowanie własne

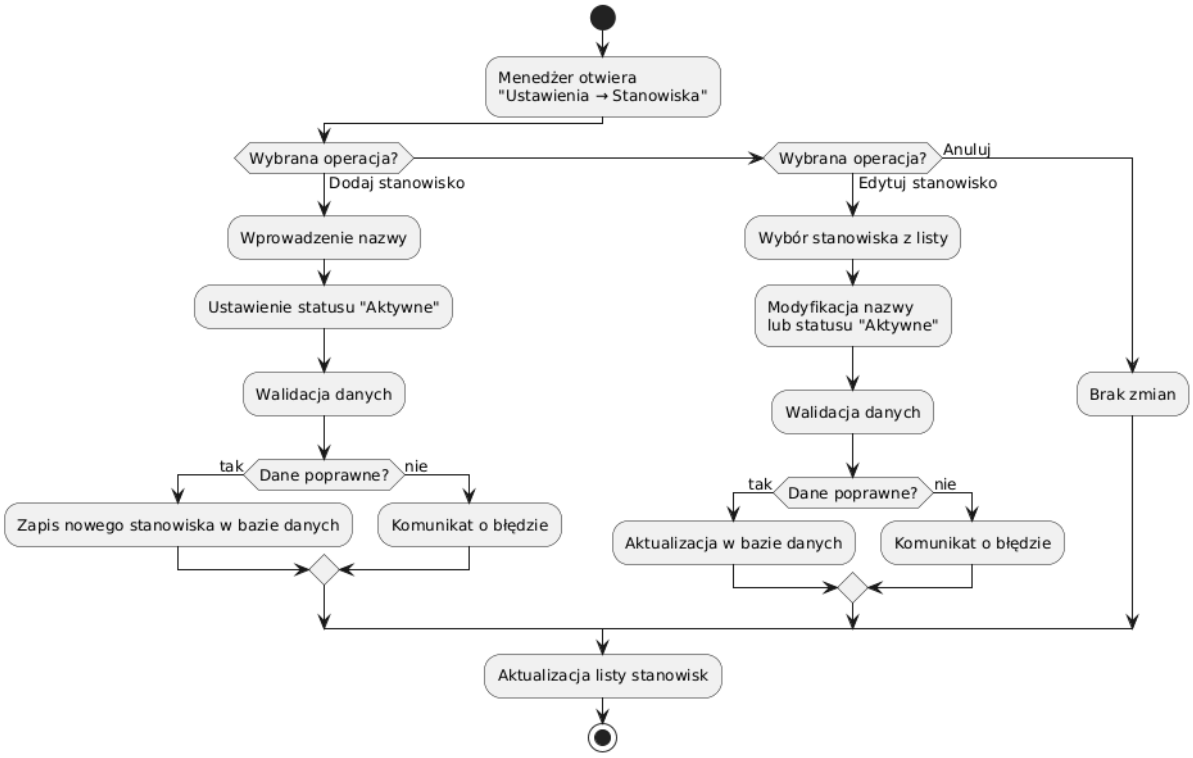
Diagram aktywności zarządzania pracownikami przedstawiony na rysunku 4.4 pokazuje operacje dodawania, edycji i dezaktywacji pracowników, a także przypisywania im stanowisk.



Rysunek 4.5: Diagram aktywności procesu definiowania zmian roboczych

Źródło: opracowanie własne

Diagram aktywności definiowania zmian roboczych przedstawiony na rysunku 4.5 pokazuje sposób określania parametrów zmian, takich jak czas trwania, liczba wymaganych pracowników oraz przypisane stanowiska.



Rysunek 4.6: Diagram aktywności procesu zarządzania stanowiskami

Źródło: opracowanie własne

Diagram aktywności procesu zarządzania stanowiskami przedstawiony na rysunku 4.6 pokazuje operacje związane z dodawaniem, edycją oraz dezaktywacją stanowisk.

Opisane procesy funkcjonalne są bezpośrednio związane z modelem matematycznym. Ograniczenia funkcjonalne systemu odpowiadają ograniczeniom formalnym zdefiniowanym w modelu optymalizacyjnym.

4.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają cechy jakościowe systemu wspomagającego planowania harmonogramów pracy, które nie są bezpośrednio związane z funkcjami, ale wpływają na ich sposób działania oraz wydajność systemu [10]. Wymagania te wpływają na komfort użytkowników aplikacji oraz praktyczne możliwości zastosowania jej w lokalu gastronomicznym.

Generowanie harmonogramu pracy powinno trwać akceptowalny okres czasu, czyli system powinien być odpowiednio wydajny. Dla danych wejściowych obejmujących kilkunastu pracowników generowanie harmonogramu na miesiąc nie powinno przekraczać kilkunastu sekund.

System powinien być skalowalny w zakresie liczby użytkowników, ilości zmian roboczych oraz okresu, na który planowany jest harmonogram. Rozwiązanie powinno umożliwiać realnie większą liczbę danych wejściowych, aby mogło być również wykorzystywane przez większe lokale gastronomiczne.

System powinien być również niezawodny, czyli mechanizmy walidacji danych wejściowych powinny działać poprawnie oraz błędy użytkownika nie mogą zostać zignorowane. W sytuacji brakujących lub niespójnych danych, system nie powinien dopuścić do uruchomienia żądanego procesu oraz poinformować o błędzie użytkownika, a w przypadku nieznalezienia rozwiązania, również powinien zwrócić użytkownikowi odpowiedni komunikat. Aplikacja jednak nie powinna się zawieszać lub przerywać swojego działania.

Istotna jest również czytelność interfejsu użytkownika. Powinien być intuicyjny oraz szybko wykonywać podstawowe operacje, takie jak tworzenie i edytowanie dyspozycyjności czy przeglądanie przypisań w harmonogramie. Prezentacja harmonogramu powinna być czytelna i jasna, czyli nie może wprowadzać użytkowników w błąd.

4.3. Projekt bazy danych

Projekt bazy danych stanowi fundamentalną część systemu planowania harmonogramu pracy, ponieważ trwale przechowuje dane, odpowiada za ich spójność oraz efektywne ich przetwarzanie przez logikę biznesową. Struktura bazy danych została zaprojektowana w sposób relacyjny z uwzględnieniem głównych zasad normalizacji danych oraz tak, aby rozdzielić dane konfiguracyjne od danych operacyjnych.

4.3.1. Główne encje systemu

Na podstawie analizy wymagań funkcjonalnych opracowano zestaw głównych encji bazy danych, które zostały przedstawione w tabeli 4.1 wraz z ich podstawowym przeznaczeniem.

Tabela 4.1: Główne encje bazy danych systemu planowania harmonogramu pracy

Encja	**Opis**
`Roles`	Słownik ról systemowych określających poziom uprawnień użytkowników w systemie.
`Users`	Użytkownicy systemu (pracownicy oraz osoby zarządzające). Encja przechowuje dane identyfikacyjne, dane logowania, przypisaną rolę oraz status aktywności konta.
`Positions`	Słownik stanowisk pracy wykorzystywany przy definiowaniu zmian oraz kwalifikacji pracowników.
`UserPositions`	Encja pośrednia realizująca relację wielu-do-wielu pomiędzy użytkownikami a stanowiskami, określająca kompetencje pracowników.
`ScheduleEntries`	Konkretne wpisy harmonogramu w danym dniu, stanowiące podstawę do przydzielania pracowników do pracy.
`Assignments`	Przydziały pracowników do wpisów harmonogramu, określające faktyczną obsadę zmian.
`AvailabilityOverrides`	Wyjątki dyspozycyjności pracowników dla konkretnych dat i przedziałów czasowych.
`Notifications`	Powiadomienia systemowe przekazywane użytkownikom, np. o publikacji harmonogramu.
`PushSubscriptions`	Dane subskrypcji powiadomień typu Web Push przypisane do użytkowników. Encja umożliwia rejestrowanie wielu urządzeń dla jednego użytkownika oraz zarządzanie aktywnością subskrypcji.
`DayNotes`	Notatki organizacyjne przypisane do konkretnych dni, wspierające planowanie pracy.

Źródło: opracowanie własne

4.3.2. Relacje pomiędzy encjami

Relacje pomiędzy encjami bazy danych zostały zaprojektowane, aby zepewnić spójność danych, jednoznaczność powiązań oraz możliwość rozwoju systemu. W modelu wykorzystano relacje typu jeden-do-wielu oraz wiele-do-wielu, implementowane przy pomocy kluczy obcych oraz tabel pośrednich. Zestawienie podstawowych relacji pomiędzy encjami wraz z ich typem zostały przedstawione w tabeli 4.2.

Tabela 4.2: Typy relacji pomiędzy głównymi encjami bazy danych

Relacja	**Typ**	**Realizacja w bazie danych**
`Roles` → `Users`	1:N	`Users.role_id` jako klucz obcy do `Roles.id`
`Users` ↔ `Positions`	N:M	Tabela pośrednia `UserPositions(user_id, position_id)`
`Positions` → `ScheduleEntries`	1:N	`ScheduleEntries.position_id` jako klucz obcy do `Positions.id`
`Users` ↔ `ScheduleEntries`	N:M	Tabela pośrednia `Assignments(entry_id, user_id)`
`Users` → `AvailabilityOverrides`	1:N	`AvailabilityOverrides.user_id` jako klucz obcy do `Users.id`
`Users` → `PushSubscriptions`	1:N	`PushSubscriptions.user_id` jako klucz obcy do `Users.id`

Źródło: opracowanie własne

4.3.3. Uzasadnienie przyjętego modelu danych

W pierwszym etapie projektowania rozdzielono dane o charakterze konfiguracyjnym od danych operacyjnych. Encje takie jak `Users`, `Roles` oraz `Positions` przechowują informacje wykorzystywane wielokrotnie w procesie planowania, natomiast `ScheduleEntries` oraz `Assignments` reprezentują konkretne harmonogramy oraz ich obsadę w danym okresie. Takie podejście ogranicza redundancję danych oraz ułatwia modyfikację konfiguracji systemu bez wpływu na dane historyczne.

Relacja wielu-do-wielu pomiędzy użytkownikami a stanowiskami pracy została zrealizowana przy użyciu tabeli pośredniej `UserPositions`. Rozwiązanie to umożliwia przypisanie pracownika do wielu stanowisk oraz obsługę wielu pracowników na tym samym stanowisku, co odpowiada rzeczywistym potrzebom organizacyjnym i pozwala na elastyczne planowanie pracy.

Encja `ScheduleEntries` zawiera informacje dotyczące daty, godzin pracy oraz stanowiska. Każdy rekord odpowiada pojedynczej zmianie w harmonogramie i stanowi bezpośrednią jednostkę planistyczną wykorzystywaną przez moduł generowania grafiku.

W modelu przyjęto możliwość istnienia wpisu harmonogramu bez przypisanego pracownika. Osiągnięto to poprzez zastosowanie osobnej encji `Assignments`, która realizuje przydział użytkowników do wpisów harmonogramu. Takie rozwiązanie umożliwia spełnienie wymagania dopuszczenia pozostawienia zmiany jako nieobsadzonej w sytuacji braku rozwiązania wykonalnego, bez naruszania integralności struktury harmonogramu.

Zastosowanie typu wyliczeniowego dla pola `AvailabilityLevel` pozwala na jednoznaczne ograniczenie dopuszczalnych wartości oraz zwiększenie czytelności danych. Jednocześnie rozwiązanie to ułatwia walidację danych po stronie bazy oraz warstwy aplikacyjnej.

Uwzględniono również encje wspierające, takie jak `AvailabilityOverrides`, `Notifications` oraz `DayNotes`, które nie wpływają bezpośrednio na strukturę harmonogramu, lecz umożliwiają obsługę wyjątków dyspozycyjności, komunikację systemową oraz przekazywanie kontekstu organizacyjnego. Ich wydzielenie pozwala zachować przejrzystość głównego modelu planistycznego.

W modelu w celu obsługi powiadomień systemowych w aplikacji typu PWA wprowadzono encję `PushSubscriptions`, przechowującą dane subskrypcji Web Push przypisane do użytkowników. Zastosowanie osobnej tabeli wynika z faktu, że pojedynczy użytkownik może korzystać z systemu na wielu urządzeniach (np. telefon oraz komputer), a każde z nich generuje odrębną subskrypcję.

Przyjęte rozwiązanie umożliwia niezależne zarządzanie aktywnością subskrypcji bez ingerencji w dane użytkownika, co ułatwia obsługę sytuacji takich jak wylogowanie, reinstalacja aplikacji, zmiana urządzenia lub cofnięcie zgód na powiadomienia. Dodatkowo rozdzielenie danych subskrypcji od encji `Users` wspiera normalizację modelu oraz ogranicza ryzyko duplikowania pól w przypadku wielu punktów końcowych dla jednego użytkownika.

4.3.4. Powiązanie bazy danych z wymaganiami funkcjonalnymi

Tabela 4.3 przedstawia powiązanie podstawowych wymagań funkcjonalnych z encjami bazy danych.

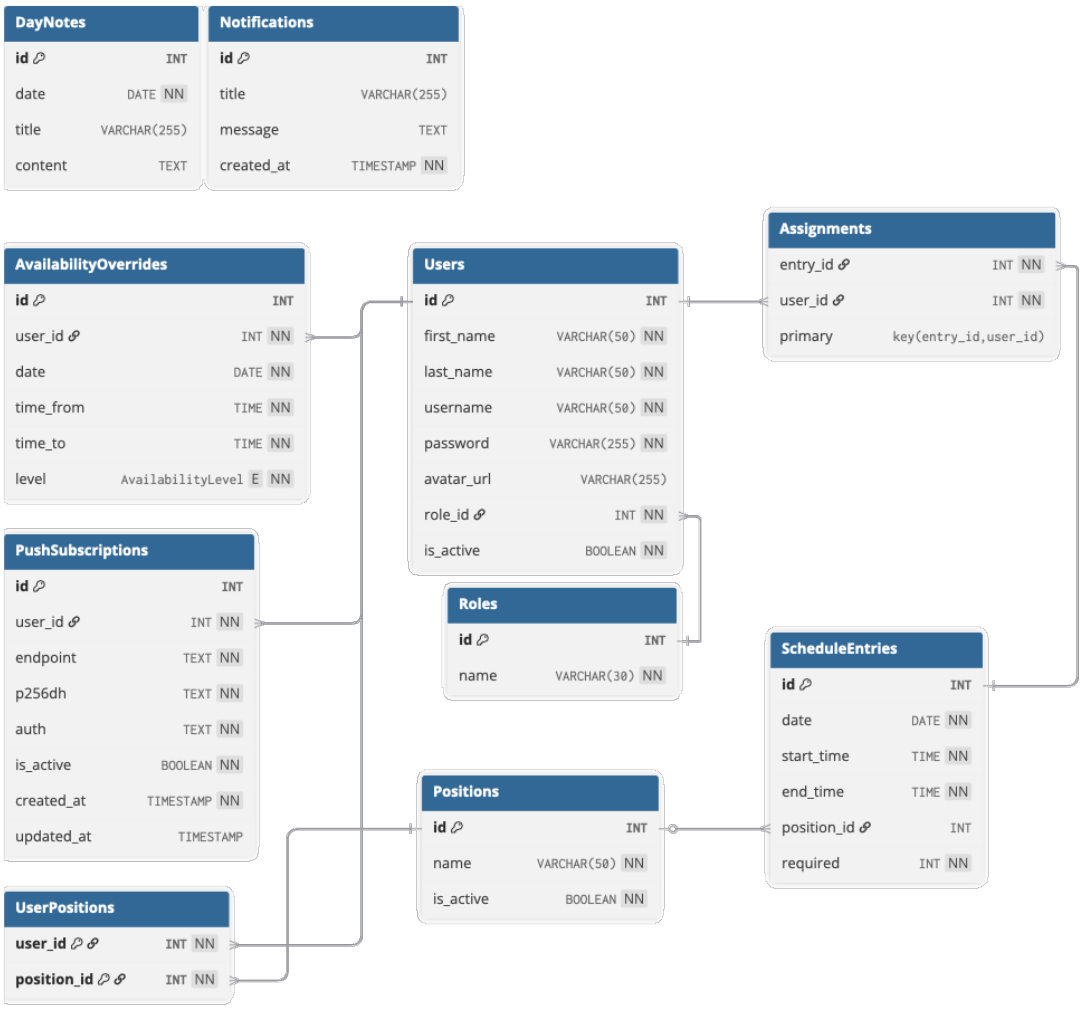
Tabela 4.3: Hasłowe powiązanie funkcjonalności systemu z encjami bazy danych

Zakres funkcjonalny	**Encje bazy danych**
Użytkownicy i role	`Users`, `Roles`
Stanowiska i kwalifikacje	`Positions`, `UserPositions`
Dyspozycyjność pracowników	`AvailabilityOverrides`
Obsada zmian	`Assignments`
Prezentacja harmonogramu	`ScheduleEntries`, `Assignments`
Informacje pomocnicze	`Notifications`, `DayNotes`, `PushSubscriptions`

Źródło: opracowanie własne

4.3.5. Schemat ERD

Diagram relacyjny bazy danych został opracowany z wykorzystaniem narzędzia *dbdiagram.io*, które umożliwia definiowanie struktur tabel w postaci tekstowej oraz automatyczne generowanie diagramów ERD. Model danych został zaprojektowany samodzielnie przez autora pracy (rysunek 4.7).



Rysunek 4.7: Schemat ERD bazy danych systemu planowania harmonogramu pracy

Źródło: opracowanie własne

4.4. Projekt API i podział modułów

Projekt API systemu planowania harmonogramu pracy został zaprojektowany w architekturze modułowej, z wyraźnym rozdziałem odpowiedzialności pomiędzy poszczególne komponenty systemu. API stanowi warstwę pośrednią pomiędzy interfejsem użytkownika a bazą danych oraz modułem planistycznym i odpowiada za realizację logiki biznesowej systemu.

Interfejs użytkownika komunikuje się z backendem poprzez API udostępniające operacje na zasobach systemu w sposób synchroniczny. Zastosowane rozwiązanie umożliwia dynamiczną zmianę widoków aplikacji po stronie klienta, bez konieczności przeładowywania strony, przy jednoczesnym zachowaniu centralnej kontroli logiki po stronie serwera.

W ramach API wyróżniono następujące logiczne moduły:

— moduł użytkowników i ról, odpowiedzialny za zarządzanie kontami oraz uprawnieniami,
— moduł konfiguracji pracy, obejmujący stanowiska, definicje zmian oraz dyspozycyjność pracowników,
— moduł harmonogramów, odpowiadający za tworzenie, edycję, publikację oraz archiwizację harmonogramów,
— moduł planistyczny, realizujący proces generowania harmonogramu na podstawie danych wejściowych oraz zdefiniowanych ograniczeń,
— moduł pomocniczy, obsługujący notatki dzienne oraz powiadomienia systemowe.

Moduł planistyczny korzysta z danych wejściowych pobieranych z bazy danych za pośrednictwem warstwy API, natomiast wyniki procesu generowania harmonogramu zapisywane są w postaci wpisów harmonogramu oraz przydziałów pracowników. Takie podejście zapewnia spójność danych oraz umożliwia kontrolę poprawności wygenerowanych wyników przed ich udostępnieniem użytkownikom.

Przyjęty podział modułów oraz sposób komunikacji pomiędzy nimi umożliwia dalszą rozbudowę systemu, w szczególności o dodatkowe funkcjonalności planistyczne lub alternatywne interfejsy użytkownika, bez konieczności istotnych zmian w istniejącej architekturze.

5. Implementacja

5.1. Backend i baza danych

Warstwa backendowa została wykonana w technologii PHP i udostępnia interfejs HTTP w postaci endpointów zwracających odpowiedzi JSON. Implementacja została rozdzielona na dwa obszary: endpointy użytkownika dostępne w katalogu `/web/api/helpers/` oraz endpoint administracyjny w `/web/api/admin-api.php`. Główne endpointy zaimplementowane w systemie zostały przedstawione w tabeli 5.1. Taki podział oddziela wykonywanie codziennych operacji (np. dyspozycyjność, notatki, profil) od operacji zarządzania użytkownikami i stanowiskami.

Tabela 5.1: Zestawienie głównych endpointów zaimplementowanych w systemie

[Dostęp: **P** - publiczny, **Z** - zalogowany, **A** - administrator]

Metoda	**Endpoint**	**Dostęp**	**Przeznaczenie**
POST	`/auth.php?action=login`	P	Logowanie użytkownika, zwrot tokenu i danych profilu
GET/POST	`/auth.php?action=me`	Z	Pobranie danych bieżącego użytkownika i odświeżenie tokenu
POST	`/auth.php?action=logout`	Z	Wylogowanie i zakończenie sesji
POST	`/auth.php?action=change-password`	Z	Zmiana hasła użytkownika
GET	`/dictionary.php`	Z	Słowniki systemowe: użytkownicy, stanowiska, mapowania
GET	`/scheduleEntries.php`	Z	Pobranie wpisów harmonogramu
GET	`/availability.php`	Z	Pobranie dyspozycyjności użytkownika
POST	`/availabilityEdit.php`	Z	Zapis dyspozycyjności dla wskazanego dnia
POST	`/plannerEntriesEdit.php`	Z	Operacje `CUD` na zmianach
POST	`/plannerAssignmentEdit.php`	Z	Przypisywanie i odpinanie pracowników od zmian
POST	`/plannerOptimize.php`	A	Automatyczny przydział zmian z użyciem solvera
GET	`/pushConfig.php`	Z	Pobranie konfiguracji Web Push (klucz publiczny VAPID)
POST	`/pushSubscribe.php`	Z	Rejestracja subskrypcji push urządzenia
POST	`/pushUnsubscribe.php`	Z	Wyrejestrowanie subskrypcji push
POST	`sendNotification.php`	A	Zapis oraz wysyłka powiadomienia do użytkowników
GET/POST	`/admin-api.php`	A	Operacje administracyjne: użytkownicy i stanowiska

Źródło: opracowanie własne

Komunikacja z bazą PostreSQL jest realizowana przez warstwę dostępu do danych opartą o zapytania SQL. W implementacji operacji modyfikujących zastosowano walidację danych wejściowych oraz kontrolę typów (m.in. format daty `YYYY-MM-DD`, format czasu czy zakresy wartości). W miejscach wieloetapowych wykorzystano transakcję, co zapewnia atomowość zmian. Przykładowo, aktualizacja użytkownika jest wykonywana jako jedna operacja transakcyjna, a w przypadku błędu następuje wycofanie wszystkich zmian.

Istotnym elementem wykonawczym jest mechanizm uwierzytelniania i autoryzacji. System obsługuje sesję serwerową oraz token Bearer (JWT), co wpływa na bezpieczeństwo w standardowym przepływie przeglądarkowym, jak przy żądaniach API. Uprawnienia są egzekwowane po stronie backendu na podstawie roli użytkownika. Dodatkowo weryfikowany jest status aktywności konta, dzięki czemu zablokowane konto nie może wykonywać operacji biznesowych.

W implementacji uwzględniono również reguły spójności harmonogramu. Podczas przypisywania pracownika do zmiany wykonywana jest kontrola kolizji czasowych, co zapobiega równoczesnemu przypisaniu tej samej osoby do nakładających się na siebie zmian.

W kontekście danych operacyjnych backend realizuje pełny cykl pracy: pobranie słowników i wpisów harmonogramu, modyfikację zmian, zarządzanie przypisaniami oraz obsługę dyspozycyjności pracowników. Osobny fragment implementacji dotyczy powiadomień push: rejestrowane są subskrypcje urządzeń, przechowywane metadane subskrypcji oraz wykonywana wysyłka powiadomień (w tym przypomnień o zmianach). Dzięki temu warstwa serwerowa nie tylko wyświetla i zapisuje dane, ale również aktywnie wspiera proces organizacji pracy.

5.2. Integracja z solverem OR-Tools

Model optymalizacyjny systemu został zaimplementowany w języku Python z wykorzystaniem biblioteki Google OR-Tools oraz solvera CP-SAT. Funkcjonuje jako niezależny komponent systemu wywołany przez backend w celu generowania optymalnych harmonogramów pracy na podstawie danych wejściowych.

Proces integracji polega na przekazaniu danych z bazy danych i informacji dodatkowych (np. priorytety przypisań, udziały wag ustalające docelową liczbę godzin) oraz na ich podstawie budowany jest model optymalizacyjny i uruchamiany solver CP-SAT. Po zakończeniu obliczeń rozwiązanie jest przekazywane do backendu i zapisywane do bazy danych, a następnie frontend prezentuje wyniki użytkownikowi.

5.2.1. Model decyzyjny i ograniczenia twarde

Model optymalizacyjny został zbudowany jako problem binarny CP-SAT. Dla każdej dopuszczalnej pary (e, u), gdzie e oznacza zmianę, a u użytkownika, tworzona jest zmienna decyzyjna. Zmienne są tworzone wyłącznie dla zmian nieoznaczonych jako `locked`, czyli takich, które wolno modyfikować w danym uruchomieniu solvera.

Ograniczenie 1: pojemność zmiany Każda zmiana ma parametr `required`. Model uwzględnia już istniejące przypisania, a solver może dołożyć tylko brakującą liczbę osób.

```
for e in entries:
```

```
    eid = str(e["id"])
    if e.get("locked"):
        continue
    required = int(e.get("required") or 1)
    existing_count = len(existing_by_entry.get(eid, set()))
    remaining = max(0, required - existing_count)
    model.Add(sum(x[(eid, uid)] for uid in user_ids) <= remaining)
```

Kod źródłowy 5.1: Ograniczenie pojemności z uwzględnieniem przypisań istniejących

Ograniczenie 2: brak duplikacji przypisania Jeżeli para (e, u) już występuje w danych wejściowych jako przypisanie stałe, solver nie może jej dodać ponownie.

```
for (eid, uid) in existing_assignments:
    if (eid, uid) in x:
        model.Add(x[(eid, uid)] == 0)
```

Kod źródłowy 5.2: Blokada duplikatów względem przypisań istniejących

Ograniczenie 3: zgodność stanowisk Przypisanie jest dopuszczalne tylko wtedy, gdy użytkownik posiada kwalifikacje wymagane przez zmianę.

```
for e in entries:
    eid = str(e["id"])
    if e.get("locked"):
        continue
    pos = e.get("position_id")
    if pos is None:
        continue
    pos = str(pos)
    for uid in user_ids:
        if pos not in positions_by_user.get(uid, set()):
            model.Add(x[(eid, uid)] == 0)
```

Kod źródłowy 5.3: Ograniczenie kompetencyjne stanowisk

Ograniczenie 4: twarda niedostępność (`no`) Jeżeli przedział zmiany nakłada się z deklaracją dyspozycyjności `no`, przypisanie jest bezwzględnie zabronione. Równolegle blokowane są kolizje ze zmianami już przypisanymi danemu użytkownikowi.

```
for e in entries:
    eid = str(e["id"])
    if e.get("locked"):
        continue
    day = str(e["date"])
    s = parse_hm(e["start_time"])
    t = parse_hm(e["end_time"])
    if s is None or t is None:
        continue
    for uid in user_ids:
        for (es, et) in existing_by_user_day.get((uid, day), []):
            if overlap(s, t, es, et):
```

```
                model.Add(x[(eid, uid)] == 0)
                break
        segs = avail_by_user_day.get((uid, day), [])
        for (a_s, a_t, level) in segs:
            if level != "no":
                continue
            if a_s is None or a_t is None:
                continue
            if overlap(s, t, a_s, a_t):
                model.Add(x[(eid, uid)] == 0)
                break
```

Kod źródłowy 5.4: Blokada przypisań kolidujących z dyspozycyjnością i istniejącym grafikiem

Ograniczenie 5: brak nakładania nowych zmian W obrębie tego samego dnia użytkownik nie może otrzymać dwóch nowych zmian o zachodzących na siebie przedziałach czasowych.

```
for day, eids in entries_by_day.items():
    for i in range(len(eids)):
        for j in range(i + 1, len(eids)):
            e1 = entry_info[eids[i]]
            e2 = entry_info[eids[j]]
            s1 = parse_hm(e1["start_time"]); t1 = parse_hm(e1["end_time"])
            s2 = parse_hm(e2["start_time"]); t2 = parse_hm(e2["end_time"])
            if s1 is None or t1 is None or s2 is None or t2 is None:
                continue
            if overlap(s1, t1, s2, t2):
                for uid in user_ids:
                    model.Add(x[(eids[i], uid)] + x[(eids[j], uid)] <= 1)
```

Kod źródłowy 5.5: Zakaz nakładania nowych przypisań tego samego użytkownika

Komentarz implementacyjny Taka konstrukcja modelu powoduje, że solver działa na zbiorze rozwiązań już zgodnych z regułami krytycznymi (kompetencje, kolizje czasowe, niedostępność, limity obsady). Dopiero na tym zbiorze uruchamiane są kolejne fazy optymalizacji jakościowej (balans obciążenia, preferencje, cele godzinowe), co ogranicza ryzyko uzyskania formalnie „optymalnego”, ale organizacyjnie niepoprawnego planu.

5.2.2. Funkcja celu i ograniczenia miękkie

Po zdefiniowaniu ograniczeń twardych model jest rozszerzany o kryteria jakościowe. W odróżnieniu od ograniczeń twardych, kryteria miękkie nie blokują rozwiązania, lecz wpływają na jego ocenę i wybór najlepszego wariantu.

W implementacji zastosowano podejście wieloetapowe, tzn. kolejne cele są optymalizowane sekwencyjnie, a wynik każdej fazy jest „zamrażany” jako warunek dla następnej. Dzięki temu priorytety biznesowe są egzekwowane w stałej kolejności.

Etap 1: minimalizacja nieobsadzonych minut Najwyższy priorytet ma pokrycie zapotrzebowania zmian. Każda nieobsadzona jednostka zapotrzebowania jest modelowana zmienną całkowitą `u_e`, a koszt liczony jest w minutach.

```
unfilled_vars = []
unfilled_minutes_terms = []
for e in entries:
    eid = str(e["id"])
    if e.get("locked"):
        continue
    required = int(e.get("required") or 1)
    existing_count = len(existing_by_entry.get(eid, set()))
    remaining = max(0, required - existing_count)
    filled = sum(x[(eid, uid)] for uid in user_ids)

    u = model.NewIntVar(0, remaining, f"unfilled_{eid}")
    model.Add(u == remaining - filled)
    unfilled_vars.append(u)

    dur = duration_by_entry.get(eid, 0)
    if dur > 0:
        unfilled_minutes_terms.append(dur * u)

total_unfilled_minutes = model.NewIntVar(0, total_required_minutes, "
    unfilled_minutes_total")
model.Add(total_unfilled_minutes == sum(unfilled_minutes_terms))
model.Minimize(total_unfilled_minutes)
```

Kod źródłowy 5.6: Konstrukcja nieobsadzonych miejsc i celu fazy 1

Etap 2: ograniczenie przydziałów dla osób z wagą 0 Po osiągnięciu najlepszego pokrycia zmian model minimalizuje łączną liczbę minut przypisanych osobom o zerowej wadze.

```
model.Add(total_unfilled_minutes == best_unfilled)

zero_weight_total = model.NewIntVar(0, total_required_minutes, "zero_weight_total")
model.Add(zero_weight_total == sum(zero_weight_terms))
model.Minimize(zero_weight_total)
```

Kod źródłowy 5.7: Minimalizacja przydziałów dla wagi 0 przy zachowaniu pokrycia

Etap 3: globalna równość obciążenia (warunkowo) Jeżeli cele godzinowe użytkowników są zbliżone, aktywowana jest dodatkowa minimalizacja rozrzutu minut:

$$\texttt{minutes_spread} = \max(\texttt{minutes_u}) - \min(\texttt{minutes_u})$$

To zapobiega skrajnemu nierównomiernemu podziałowi pracy.

```
model.Add(zero_weight_total == best_zero_weight)
if spread_enabled:
    model.Minimize(minutes_spread)
```

Kod źródłowy 5.8: Warunkowa minimalizacja globalnego spreadu obciążenia

Etap 4: proporcjonalność względem targetów Dla każdego użytkownika wyznaczany jest wskaźnik znormalizowany `minutes/target`, a celem jest minimalizacja rozpiętości tych wskaźników:

$$\min\left(\max(\texttt{norm_ratio_u}) - \min(\texttt{norm_ratio_u})\right)$$

W praktyce stabilizuje to proporcje przydziału przy różnych etatach i wagach.

```
model.Minimize(norm_ratio_spread)
status = solver.Solve(model)
best_norm_ratio_spread = solver.Value(norm_ratio_spread)
model.Add(norm_ratio_spread == best_norm_ratio_spread)
```

Kod źródłowy 5.9: Minimalizacja rozpiętości relacji minutes/target

Etap 5: ochrona targetów jawnych Dla użytkowników z jawnie podanym celem godzinowym najpierw minimalizowane są przekroczenia ponad target, aby nie dokładać godzin tam, gdzie nie jest to potrzebne.

```
if explicit_cap_vars:
    explicit_cap_sum = sum(explicit_cap_vars)
    model.Minimize(explicit_cap_sum)
    status = solver.Solve(model)
    best_explicit_cap_sum = int(round(solver.ObjectiveValue()))
    model.Add(explicit_cap_sum == best_explicit_cap_sum)
```

Kod źródłowy 5.10: Priorytet dla użytkowników z explicit target

Etap 6: fairness godzinowy Po ustaleniu kluczowych priorytetów uruchamiany jest cel fairness oparty o odchylenia absolutne i względne od targetów oraz spójność w grupach o tej samej wadze.

```
balance_expr = (
    (10000 * max_rel_dev_var)
    + (100 * sum(rel_dev_vars))
    + (500 * max_dev_var)
    + (5 * sum(dev_vars))
    + (100 * sum(same_weight_spreads))
)
model.Minimize(balance_expr)
```

Kod źródłowy 5.11: Funkcja fairness (balans godzin)

Etap 7: jakość końcowa harmonogramu W ostatnim kroku fairness jest utrzymywany w dopuszczalnym przedziale (slack), a optymalizowane są kryteria jakościowe:

— `pref_terms`: preferencje dyspozycyjności (`yes` lepsze od `maybe`, lepsze od `unknown`),
— `week_dev_vars`: wyrównanie obciążenia między tygodniami,
— `rest_vars`: kara za sekwencję późna-zmiana → wczesna-zmiana następnego dnia,
— `weighted_caps`, `weighted_bands`: kontrola przekroczeń i pasm tolerancji.

```
balance_slack = max(2000, total_required_minutes // 20)
model.Add(balance_expr <= best_balance + balance_slack)

secondary_expr = (
    w_cap * sum(weighted_caps)
    + w_band * sum(weighted_bands)
    + w_week * sum(week_dev_vars)
    + w_pref * sum(pref_terms)
    + w_rest * sum(rest_vars)
)
model.Minimize(secondary_expr)
```

Kod źródłowy 5.12: Końcowa funkcja jakości z utrzymaniem fairness

Interpretacja wag i stabilność modelu Wagi (`w_cap`, `w_band`, `w_pref`, `w_week`, `w_rest`) zostały dobrane eksperymentalnie i mają charakter regulacyjny. Kluczowe jest to, że dzięki sekwencyjnej optymalizacji:

1. najpierw spełniany jest warunek pokrycia zmian,
2. następnie chronione są priorytety godzinowe,
3. dopiero na końcu poprawiana jest „jakość miękka” harmonogramu.

Takie podejście minimalizuje ryzyko sytuacji, w której poprawa jednego kryterium estetycznego pogarsza kryterium krytyczne biznesowo.

5.3. Interfejs użytkownika

Interfejs został zaimplementowany jako responsywna aplikacja webowa (RWD), dostępna na komputerach i urządzeniach mobilnych, z naciskiem na szybki dostęp do najczęściej używanych funkcji planowania harmonogramów. Interfejs przygotowano w dwóch głównych perspektywach pracy, aby uprościć obsługę aplikacji oraz ograniczyć ryzyko błędów: administracyjnej (planowanie i edycja harmonogramu, zarządzanie użytkownikami i stanowiskami) oraz użytkownika końcowego (podgląd zmian, dyspozycyjność, powiadomienia).

Interfejs jest responsywny i poprawnie skaluje się do ekranów desktopowych oraz mobilnych. W praktyce widoki administracyjne są najwygodniejsze na laptopie, natomiast widok pracownika jest zoptymalizowany pod ekran smarfona, gdzie kluczowa jest szybkość dostępu do danych oraz wygodne zgłaszanie dyspozycyjności.

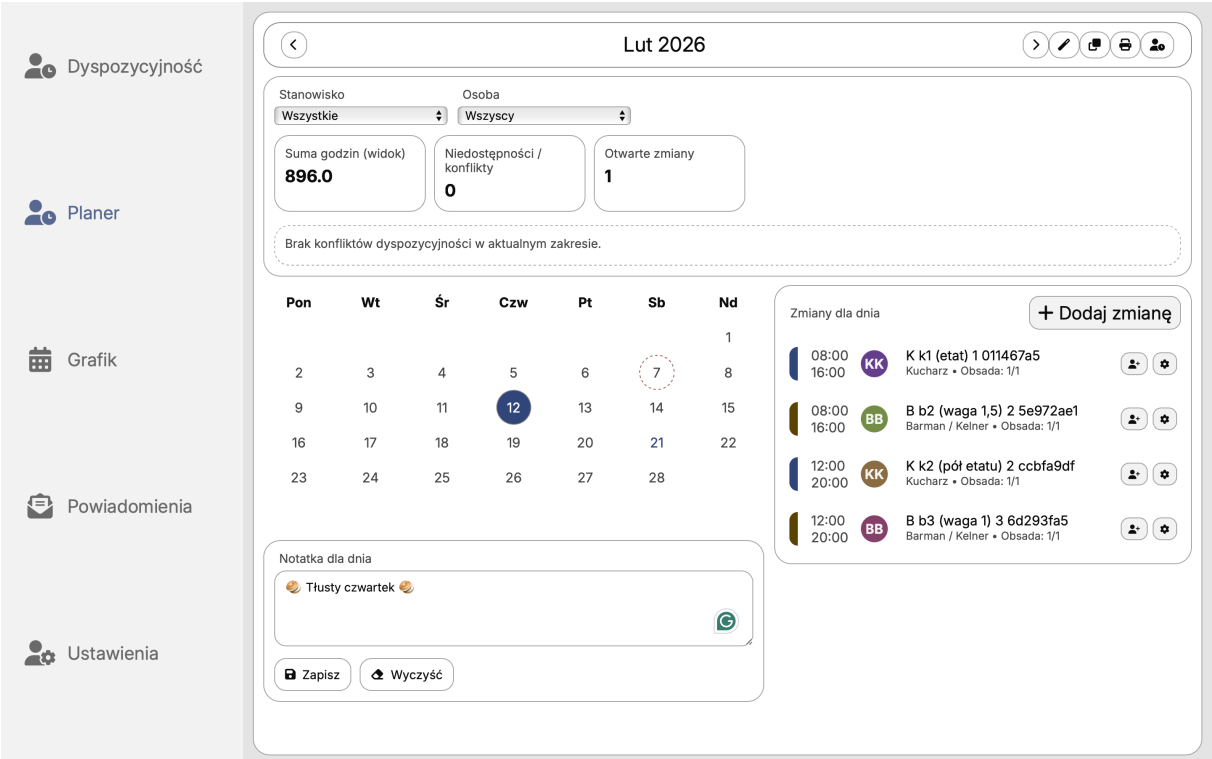
W aplikacji zaimplementowano mechanizm odświeżania danych bez konieczności przeładowywania całej strony. Dane (m.in. wpisy grafiku, notatki) są pobierane asynchronicznie z endpointów API, a następnie aktualizowane w wybranych komponentach interfejsu. Takie rozwiązanie skraca czas reakcji oraz poprawia komfort pracy, szczególnie przy częstych zmianach w aplikacji.

Istotnym elementem implementacji jest również wsparcie PWA (ang. *Progressive Web App*). Aplikacja posiada manifest, serwis workera oraz możliwość dodania aplikacji do ekranu głównego (jako aplikacji mobilnej). Dzięki temu interfejs działa w trybie zbliżonym do aplikacji natywnej, a dodatkowo wspiera mechanizm powiadomień push.

Zastosowano również obsługę motywu jasnego i ciemnego, co poprawia ergonomię użytkownika w różnych warunkach oświetleniowych. Zachowana została spójność typografii, kolorystyki

stanów interaktywnych oraz czytelność kluczowych elementów interfejsu (m.in. kalendarz, lista zmian).

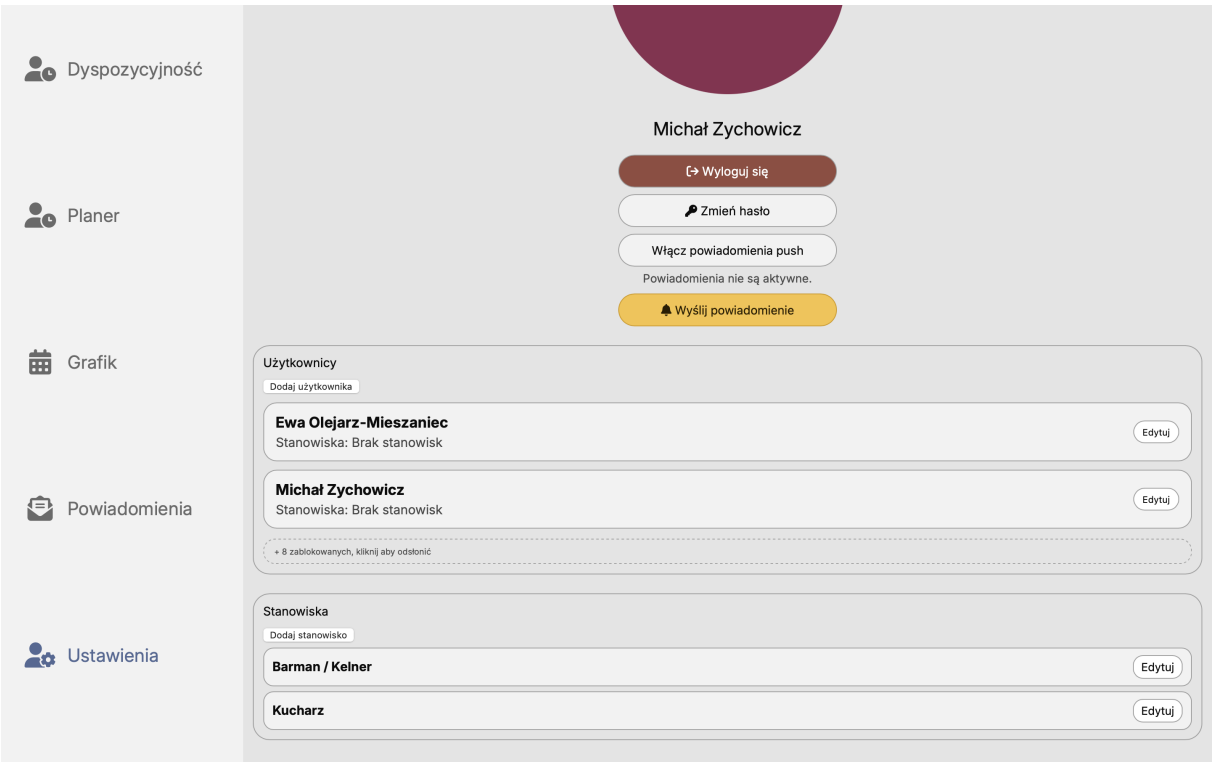
5.3.1. Opis głównych ekranów



Rysunek 5.1: Widok laptopowy administratora: planer zmian

Źródło: opracowanie własne

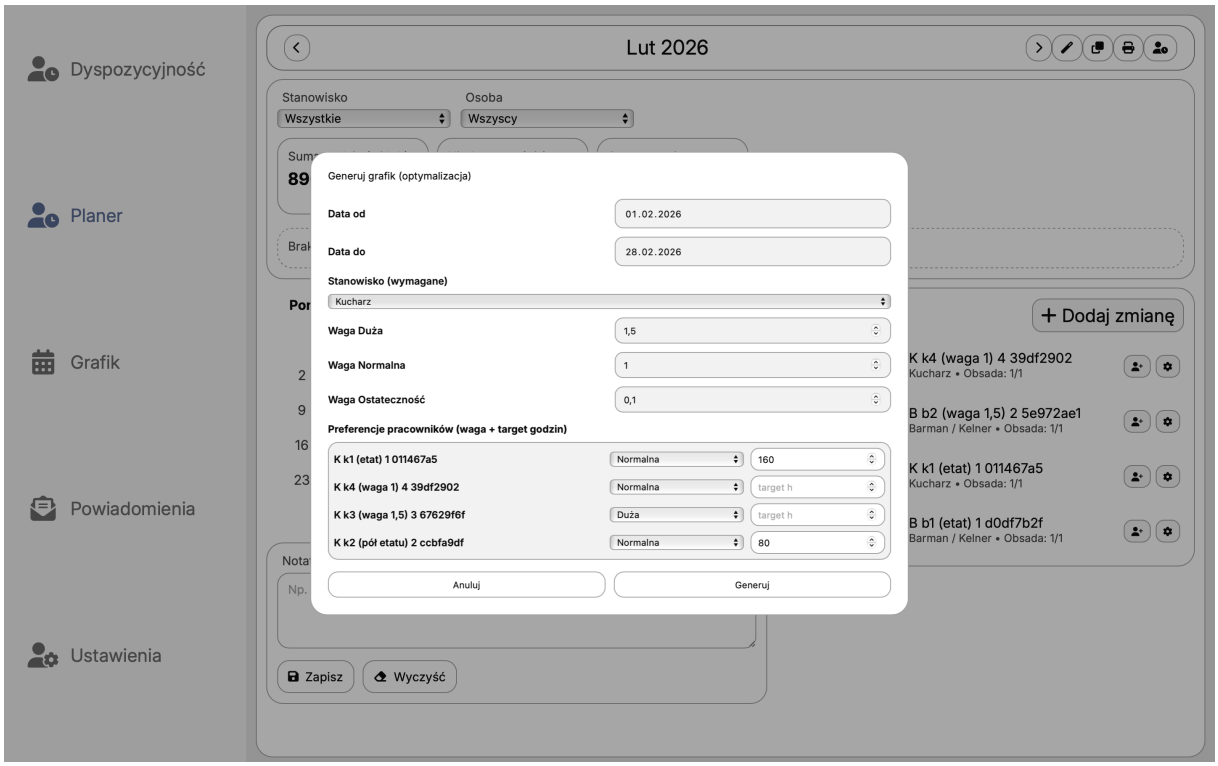
Ekran planera (rys. 5.1) prezentuje przegląd kalendarza miesięcznego, podsumowanie liczby godzin, identyfikację konfliktów dyspozycyjności oraz zarządzanie zmianami dla wybranego dnia. Administrator może ręcznie dodawać zmiany, przypisywać pracowników oraz generować grafik z wykorzystaniem modułu optymalizacyjnego.



Rysunek 5.2: Widok laptopowy administratora: ustawienia (użytkownicy i stanowiska)

Źródło: opracowanie własne

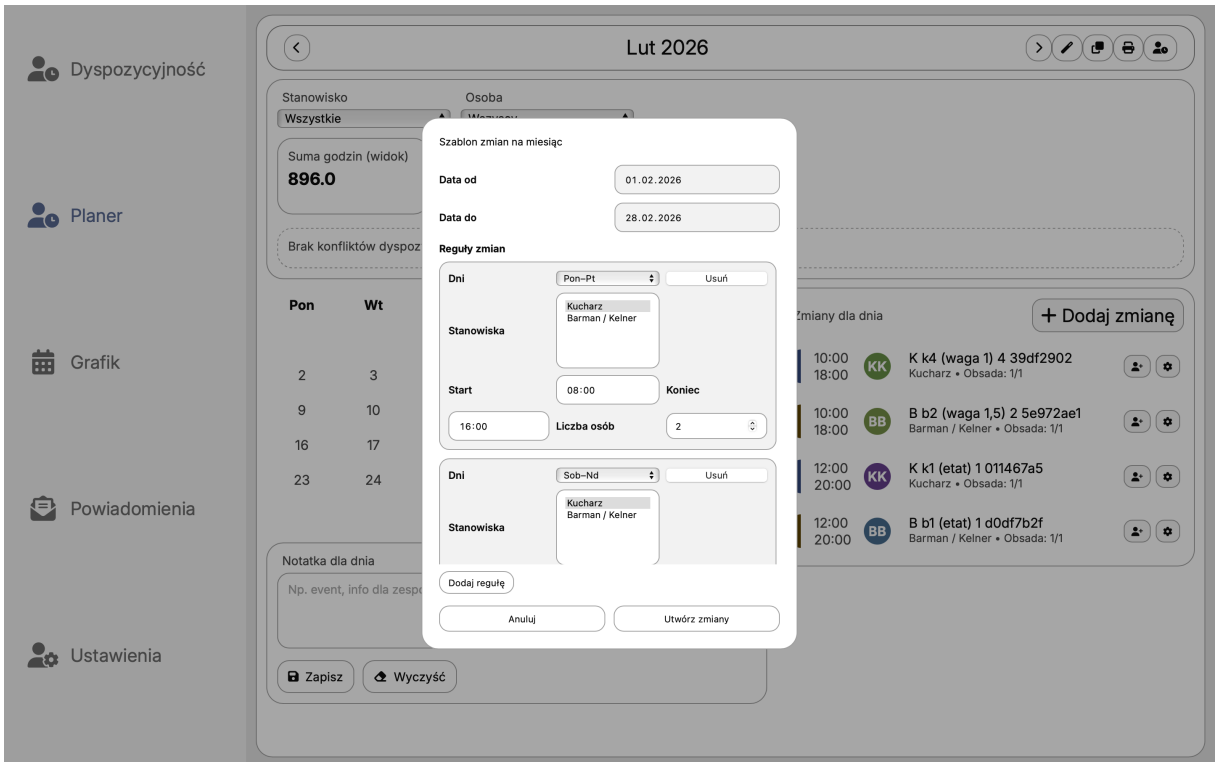
Ekran panelu ustawień przedstawiony na rysunku 5.2 prezentuje moduł administracyjny odpowiedzialny za zarządzanie użytkownikami systemu oraz definicjami stanowisk. Administrator może dodawać, edytować oraz blokować konta użytkowników, a także przypisywać im stanowiska pracy.



Rysunek 5.3: Widok laptopowy administratora: przygotowanie danych do solvera

Źródło: opracowanie własne

Ekran konfiguracji parametrów optymalizacji (rys. 5.3) prezentuje formularz uruchamiania procesu optymalizacji harmonogramu. Administrator określa zakres dat, wymagane stanowiska oraz parametry wag (duża, normalna, ostateczność), które wpływają na działanie solvera. Dodatkowo możliwe jest zdefiniowanie preferencji pracowników, w tym wag indywidualnych oraz docelowej liczby godzin (target). Parametry te są wykorzystywane w modelu optymalizacyjnym do minimalizacji odchyleń od oczekiwanego rozkładu godzin oraz do rozstrzygania konfliktów przydziałów.



Rysunek 5.4: Widok laptopowy administratora: projektowanie zmian i reguł

Źródło: opracowanie własne

Ekran definiowania szablonu zmian przedstawiony na rysunku 5.4 prezentuje moduł tworzenia szablonu zmian dla wybranego zakresu dat. Administrator może określić reguły generowania zmian w podziale na dni tygodnia, stanowiska, godziny rozpoczęcia i zakończenia pracy oraz wymaganą liczbę osób na danym stanowisku. Zastosowanie reguł cyklicznych umożliwia szybkie wygenerowanie struktury zmian dla np. całego miesiąca, bez konieczności ręcznego wprowadzania każdej zmiany osobno. Funkcjonalność ta znacząco skraca czas przygotowania harmonogramu bazowego.

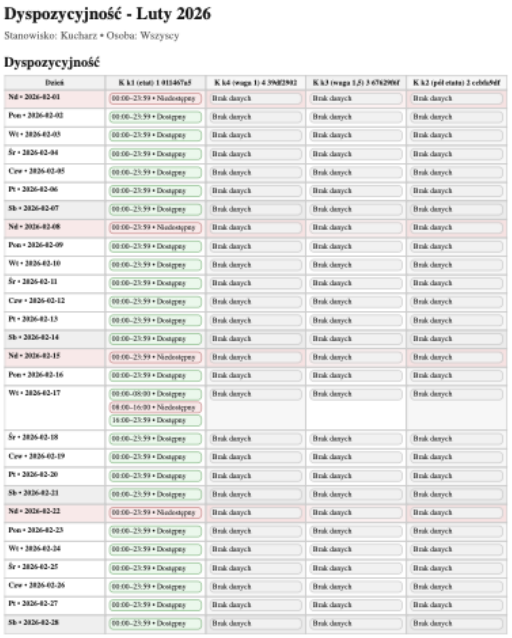
Grafik - Luty 2026

Stanowisko: Kucharz • Osoba: Wszyscy

Zmiany

Dzień	K k1 (etat) 1 015467a5	K k4 (waga 1) 4 59df2902	K k3 (waga 1,5) 3 676290bf	K k2 (pół etatu) 2 cebfa9df
Nd • 2026-02-01			12:00–20:00 (8h) • Kucharz	10:00–18:00 (8h) • Kucharz
Pon • 2026-02-02	08:00–16:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Wt • 2026-02-03	08:00–16:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Śr • 2026-02-04	12:00–20:00 (8h) • Kucharz			08:00–16:00 (8h) • Kucharz
Czw • 2026-02-05	12:00–20:00 (8h) • Kucharz		08:00–16:00 (8h) • Kucharz	
Pt • 2026-02-06	08:00–16:00 (8h) • Kucharz	12:00–20:00 (8h) • Kucharz		
Sb • 2026-02-07	12:00–20:00 (8h) • Kucharz			10:00–18:00 (8h) • Kucharz
Nd • 2026-02-08		12:00–20:00 (8h) • Kucharz	10:00–18:00 (8h) • Kucharz	
Pon • 2026-02-09		12:00–20:00 (8h) • Kucharz	08:00–16:00 (8h) • Kucharz	
Wt • 2026-02-10		08:00–16:00 (8h) • Kucharz	12:00–20:00 (8h) • Kucharz	
Śr • 2026-02-11	12:00–20:00 (8h) • Kucharz	08:00–16:00 (8h) • Kucharz		
Czw • 2026-02-12	08:00–16:00 (8h) • Kucharz			12:00–20:00 (8h) • Kucharz
Pt • 2026-02-13	08:00–16:00 (8h) • Kucharz			12:00–20:00 (8h) • Kucharz
Sb • 2026-02-14	10:00–18:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Nd • 2026-02-15		10:00–18:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz
Pon • 2026-02-16	12:00–20:00 (8h) • Kucharz		08:00–16:00 (8h) • Kucharz	
Wt • 2026-02-17	08:00–16:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Śr • 2026-02-18	08:00–16:00 (8h) • Kucharz	12:00–20:00 (8h) • Kucharz		
Czw • 2026-02-19	08:00–16:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Pt • 2026-02-20			12:00–20:00 (8h) • Kucharz	08:00–16:00 (8h) • Kucharz
Sb • 2026-02-21	12:00–20:00 (8h) • Kucharz	10:00–18:00 (8h) • Kucharz		
Nd • 2026-02-22			10:00–18:00 (8h) • Kucharz	12:00–20:00 (8h) • Kucharz
Pon • 2026-02-23	12:00–20:00 (8h) • Kucharz			08:00–16:00 (8h) • Kucharz
Wt • 2026-02-24	12:00–20:00 (8h) • Kucharz	08:00–16:00 (8h) • Kucharz		
Śr • 2026-02-25	08:00–16:00 (8h) • Kucharz			12:00–20:00 (8h) • Kucharz
Czw • 2026-02-26		08:00–16:00 (8h) • Kucharz	12:00–20:00 (8h) • Kucharz	
Pt • 2026-02-27	08:00–16:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	
Sb • 2026-02-28	10:00–18:00 (8h) • Kucharz		12:00–20:00 (8h) • Kucharz	

(a) Wydruk zmian



Dyspozycyjność - Luty 2026

Stanowisko: Kucharz • Osoba: Wszyscy

Dyspozycyjność

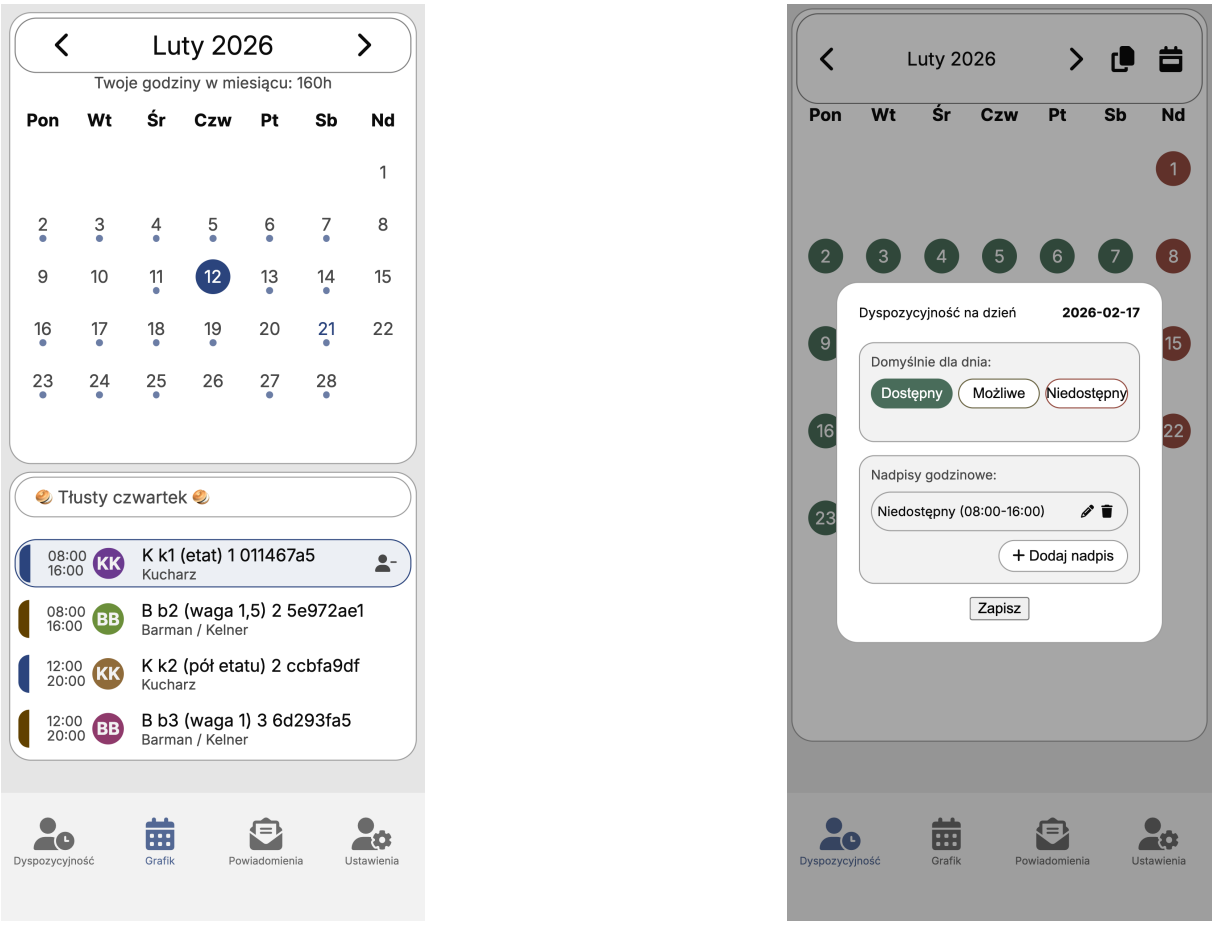
Dzień	K k1 (etat) 1 015467a5	K k4 (waga 1) 4 59df2902	K k3 (waga 1,5) 3 676290bf	K k2 (pół etatu) 2 cebfa9df
Nd • 2026-02-01	00:00–23:59 • Niedostępny	Brak danych	Brak danych	Brak danych
Pon • 2026-02-02	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Wt • 2026-02-03	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Śr • 2026-02-04	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Czw • 2026-02-05	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Pt • 2026-02-06	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Sb • 2026-02-07	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Nd • 2026-02-08	00:00–23:59 • Niedostępny	Brak danych	Brak danych	Brak danych
Pon • 2026-02-09	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Wt • 2026-02-10	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Śr • 2026-02-11	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Czw • 2026-02-12	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Pt • 2026-02-13	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Sb • 2026-02-14	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Nd • 2026-02-15	00:00–23:59 • Niedostępny	Brak danych	Brak danych	Brak danych
Pon • 2026-02-16	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Wt • 2026-02-17	00:00–08:00 • Dostępny 08:00–16:00 • Niedostępny 16:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Śr • 2026-02-18	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Czw • 2026-02-19	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Pt • 2026-02-20	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Sb • 2026-02-21	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Nd • 2026-02-22	00:00–23:59 • Niedostępny	Brak danych	Brak danych	Brak danych
Pon • 2026-02-23	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Wt • 2026-02-24	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Śr • 2026-02-25	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Czw • 2026-02-26	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Pt • 2026-02-27	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych
Sb • 2026-02-28	00:00–23:59 • Dostępny	Brak danych	Brak danych	Brak danych

(b) Wydruk dyspozycyjności

Rysunek 5.5: Widoki raportowe

Źródło: opracowanie własne

Raport zmian (rys. 5.5a) prezentuje wygenerowany harmonogram pracy w formie tabelarycznej. Każda komórka zawiera informację o godzinach pracy oraz stanowisku przypisanym danemu pracownikowi. Raport umożliwia szybkie sprawdzenie kompletności obsady oraz analizę rozkładu godzin pracy w danym okresie rozliczeniowym. Raport dyspozycyjności (rys. 5.5b) prezentuje zbiorcze zestawienie dyspozycyjności pracowników w ujęciu miesięcznym w formie tabelarycznej. W komórkach tabeli prezentowany jest status dostępności wraz z ewentualnymi przedziałami godzinowymi. Kolorystyka komórek umożliwia szybkie rozróżnienie dostępności (np. zielony - dostępny, czerwony – niedostępny).



(a) Grafik użytkownika

(b) Dyspozycyjność + okno modalne

Rysunek 5.6: Widoki mobilne użytkownika

Źródło: opracowanie własne

Ekran grafiku użytkownika (rys. 5.6a) prezentuje mobilny interfejs przeglądania harmonogramu pracy w ujęciu miesięcznym. Górna część ekranu umożliwia nawigację pomiędzy miesiącami oraz wyświetla podsumowanie liczby zaplanowanych godzin w danym okresie. Kalendarz miesięczny sygnalizuje dni z przypisanymi zmianami za pomocą znaczników graficznych. Po wybraniu konkretnego dnia użytkownik otrzymuje notatkę dnia oraz szczegółową listę zmian zaplanowanych na dany dzień. Ekran zarządzania dyspozycyjności (rys. 5.6b) prezentuje moduł definiowania dyspozycyjności pracownika. Użytkownik może określić domyślny status dostępności dla całego dnia (dostępny, możliwe, niedostępny), a także dodać szczegółowe nadpisy godzinowe (tzw. override), które modyfikują ogólną dostępność w wybranym przedziale czasowym. Zastosowane rozwiązanie pozwala na elastyczne odwzorowanie rzeczywistych ograniczeń czasowych pracownika. Mechanizm nadpisów ma pierwszeństwo względem ustawienia domyślnego dla dnia, co umożliwia precyzyjne określenie dostępności w konkretnych godzinach.

6. Testy i ewaluacja

Przyjęcie jednolitych założeń dla testów umożliwiło obiektywną ocenę wydajności i jakości generowanych harmonogramów. Wszystkie testy przeprowadzono dla wspólnych założeń obejmujących:
— 13 pracowników (4 kucharzy, 9 baristów),
— 2 stanowiska pracy (kucharz, barista),
— 4 zmiany w dni robocze (po 2 na każde stanowisko),
— 5 zmian w weekendy (3 na kucharza, 2 na baaristę),
— miesięczny zakres planowania (ok. 120 zmian).

Przeprowadzone scenariusze testowe potwierdziły poprawność działania systemu w typowych warunkach eksploatacyjnych oraz w sytuacjach wyjątkowych.

6.1. Scenariusze testowe systemu

W celu weryfikacji działania systemu opracowano zestaw scenariuszy testowych odzwierciedlających realne użycie aplikacji w środowisku gastronomicznym.

Scenariusz 1 - Logowanie użytkownika
Cel testu: Weryfikacja poprawności procesu uwierzytelniania oraz inicjalizacji sesji użytkownika.
Warunki początkowe: Użytkownik posiada aktywne konto w systemie.
Kroki testowe:
1. Wprowadzenie poprawnego loginu i hasła.
2. Wysłanie danych do backendu.
3. Weryfikacja danych w bazie.
4. Przekierowanie do odpowiedniego widoku (pracowniczego lub administracyjnego).

Rezultat:
— poprawne uwierzytelnienie użytkownika,
— utworzenie aktywnej sesji,
— załadowanie kontekstu bez błędów.

Scenariusz 2 - Zarządzanie pracownikami i stanowiskami
Cel testu: Weryfikacja poprawności operacji administracyjnych związanych z dodawaniem, edycją i dezaktywacją pracowników oraz stanowisk pracy.
Warunki początkowe: Użytkownik posiada uprawnienia menedżera.

Kroki testowe:

1. Dodanie nowego pracownika wraz z przypisaniem roli.
2. Edycja danych istniejącego pracownika.
3. Dezaktywacja pracownika.
4. Dodanie nowego stanowiska pracy.
5. Edycja istniejącego stanowiska.
6. Dezaktywacja stanowiska.
7. Przypisanie pracownika do stanowiska.

Rezultat:

— CUD pracowników i stanowisk działa poprawnie i dane są aktualizowane na bieżąco,
— dezaktywowany pracownik/stanowisko nie jest uwzględniane podczas planowania harmonogramu,
— nowo utworzone stanowiska są dostępne podczas definiowania zmian.

Scenariusz 3 - Wprowadzanie i zapis dyspozycyjności

Cel testu: Sprawdzenie poprawności zapisu danych do bazy danych oraz ich ponownego odczytu w interfejsie.

Warunki początkowe: Użytkownik jest zalogowany.

Kroki testowe:

1. Wybór dnia w kalendarzu.
2. Określenie statusu dostępności.
3. Ewentualne dodanie nadpisu godzinowego.
4. Zapis danych bez przeładowania strony.
5. Odświeżenie widoku.

Rezultat:

— poprawny zapis w bazie oraz zachowanie danych po ponownym odczycie,
— dynamiczna aktualizacja interfejsu.

Scenariusz 4 - Definiowanie zmian roboczych

Cel testu: Weryfikacja poprawności procesu tworzenia i edycji zmian roboczych w planerze.

Warunki początkowe:

— Użytkownik posiada uprawnienia menedżera.
— W systemie istnieją stanowiska pracy.

Kroki testowe:

1. Administrator wybiera zakres dat.
2. W oknie definiowania szablonu określa reguły cykliczne.
3. Uruchamia generowanie zmian na podstawie szablonu.

Rezultat:

— system generuje strukturę zmian dla całego wskazanego okresu zgodnie z zdefiniowanymi regułami,
— wygenerowane zmiany posiadają poprawnie przypisane parametry,
— nie występują duplikaty ani niespójności danych,
— wygenerowane zmiany są otwarte.

Scenariusz 5 - Generowanie harmonogramu

Cel testu: Weryfikacja komunikacji pomiędzy backendem, solverem optymalizacyjnym oraz warstwą prezentacji.

Warunki początkowe: Wprowadzona dyspozycyjność przynajmniej części pracowników.

Kroki testowe:

1. Administrator ustawia zakres planowania.
2. System generuje model optymalizacyjny.
3. Uruchomienie solvera.
4. Odbiór wyniku w formacie JSON.
5. Prezentacja harmonogramu w interfejsie.

Rezultat:

— brak błędów komunikacji,
— brak nakładających się zmian,
— przypisania zgodne z dyspozycyjnością,
— oznaczenie ewentualnych zmian nieobsadzonych jako otwarte.

Scenariusz 6 - Obsługa sytuacji braku rozwiązania

Cel testu: Sprawdzenie zachowania systemu w przypadku zbyt restrykcyjnych ograniczeń.

Warunki początkowe: Ograniczona dyspozycyjność pracowników uniemożliwiająca pełne pokrycie zmian.

Kroki testowe:

1. Wprowadzenie sprzecznych ograniczeń.
2. Uruchomienie solvera.

Rezultat:

— brak błędu systemowego,
— wygenerowanie częściowego harmonogramu,
— oznaczenie zmian nieobsadzonych jako otwarte.

Scenariusz 7 - Oddanie zmiany oraz zapisanie się do zmiany otwartej

Cel testu: Weryfikacja poprawności mechanizmu oddawania przypisanej zmiany oraz zapisywania się innego użytkownika na zmianę oznaczoną jako otwarta.

Warunki początkowe:

— Wygenerowano harmonogram.
— Użytkownik A posiada przypisaną zmianę w harmonogramie.
— Użytkownicy są zalogowani.

Kroki testowe:

1. Użytkownik oddaje przypisaną zmianę.
2. System oznacza zmianę jako otwartą.
3. System wysyła powiadomienie do użytkowników, którzy mogą zapisać się do zmiany.
4. Użytkownik B przypisuje się do zmiany.
5. System aktualizuje przypisanie w bazie danych.
6. Widok grafiku aktualizuje się bez przeładowania strony.

Rezultat:
— zmiana zostaje poprawnie oznaczona jako otwarta po rezygnacji użytkownika A,
— powiadomienie zostaje wysłane do użytkowników spełniających kwalifikacje zmiany,
— nie dochodzi do nakładania godzin ani naruszenia ograniczeń twardych,
— aktualizacja widoku następuje dynamicznie bez przeładowania strony, bez utraty danych.

6.2. Testy funkcjonalne

Testy przeprowadzono manualnie dla wymagań funkcjonalnych opisanych w podrozdziale 4.1, realizując scenariusze testowe opisane w podrozdziale 6.1.

Tabela 6.1: Powiązanie wymagań funkcjonalnych ze scenariuszami testowymi

Wymaganie funkcjonalne	**Scenariusz testowy**
Obsługa użytkowników z podziałem na role	Scenariusz 1
Zarządzanie pracownikami i stanowiskami	Scenariusz 2
Wprowadzanie i modyfikowanie dyspozycyjności	Scenariusz 3
Definiowanie zmian roboczych	Scenariusz 4
Uruchamianie procesu generowania harmonogramu	Scenariusz 5
Prezentacja harmonogramu w interfejsie	Scenariusz 5
Obsługa braku pełnego rozwiązania	Scenariusz 6
Oddanie zmiany i zapis na zmianę otwartą	Scenariusz 7
Zapis i odczyt danych w systemie	Wszystkie scenariusze

Źródło: opracowanie własne

Przeprowadzona weryfikacja potwierdziła, że wszystkie zdefiniowane wymagania funkcjonalne zostały poprawnie zaimplementowane oraz działają zgodnie z przyjętymi założeniami projektowymi.

6.2.1. Analiza poprawności przepływu danych

W ramach testów zweryfikowano poprawność komunikacji pomiędzy warstwą frontendową (JavaScript), backendem (API) oraz modułem optymalizacyjnym. W trakcie testów nie zaobserwowano:
— utraty danych podczas przesyłania JSON,
— niespójności zapisów w bazie danych,
— błędów sesji przy równoczesnym korzystaniu z systemu przez wielu użytkowników.

System poprawnie obsługiwał przypadki niekompletnych danych wejściowych (np. brak dyspozycyjności części pracowników) umożliwiając uruchomienie procesu generowania harmonogramu z ryzykiem niespełnienia oczekiwań tej grupy pracowników.

6.2.2. Stabilność działania

Podczas testów system zachowywał stabilność działania w trakcie:
— jednoczesnego wprowadzania dyspozycyjności przez wielu użytkowników,
— wielokrotnego uruchamiania solvera w krótkich odstępach czasu,
— modyfikacji harmonogramu po wygenerowaniu rozwiązania.
Nie zaobserwowano awarii ani krytycznych błędów systemowych.

6.3. Analiza wydajności algorytmu planowania

Celem przeprowadzanych testów było również określenie czasu działania solvera optymalizacyjnego w warunkach rzeczywistego wykorzystania systemu oraz ocena skalowalności rozwiązania.

Średni czas generowania harmonogramu dla wskazanej konfiguracji wynosił około 60 sekund.

Tabela 6.2: Czas działania solvera dla różnych rozmiarów danych

Liczba pracowników	**Liczba zmian**	**Czas generowania**
5	∼40	8–12 s
9	∼80	25–35 s
13	∼120	50–70 s

Źródło: opracowanie własne

Zaobserwowano wzrost czasu obliczeń wraz ze wzrostem zmiennych decyzyjnych oraz ograniczeń modelu. Charakter wzrostu jest nieliniowy, co jest zgodne z właściwościami problemów harmonogramowania należących do klasy problemów NP-trudnych [11].

Dla tych samych zestawów danych wejściowych solver respektował ograniczenia twarde oraz generował jakościowo spójne harmonogramy. W przypadku braku pełnego rozwiązania system nie generował błędów krytycznych, tylko oznaczał nieobsadzone zmiany jako otwarte.

6.4. Porównanie planowania ręcznego i automatycznego

W celu oceny użyteczności opracowanego systemu przeprowadzono porównanie procesu tworzenia harmonogramu metodą ręczną oraz przy wykorzystaniu solvera optymalizacyjnego.

Planowanie ręczne w analizowanym środowisku odbywało się na podstawie dyspozycyjności przesyłanej przez pracowników w różnych formach (wiadomości prywatne, komunikatory, notatki tekstowe). Proces był realizowany przez dwie osoby odpowiedzialne za grafik.

Średni czas przygotowania harmonogramu wynosił około 2 godzin. Dodatkowo proces wymagał:
— manualnej analizy dostępności każdego pracownika,
— weryfikacji konfliktów czasowych,
— wielokrotnego nanoszenia poprawek,
— interpretacji niejednoznacznie określonej dyspozycyjności.

W praktyce prowadziło to do błędów wynikających z niejednoznacznego zrozumienia dostępności pracowników oraz nierównomiernego rozłożenia zmian.

W przypadku planowania automatycznego każdy pracownik wprowadzał dyspozycyjność bezpośrednio do systemu, co eliminowało problem rozproszenia informacji.

Proces generowania harmonogramu sprowadzał się do:

1. wprowadzenia danych wejściowych,
2. uruchomienia solvera,
3. ewentualnej korekty ręcznej.

Tabela 6.3: Porównanie planowania ręcznego i automatycznego

Kryterium	**Planowanie ręczne**	**Planowanie automatyczne**
Czas przygotowania	ok. 2 godziny	ok. 1 minuta
Ryzyko błędnej interpretacji dyspozycyjności	wysokie	minimalne
Liczba konfliktów zmian	sporadyczne przypadki	brak (ograniczenia twarde)
Równomierność rozłożenia zmian	zależna od osoby planującej	kontrolowana przez model
Obciążenie poznawcze	wysokie	niskie

Źródło: opracowanie własne

Przeprowadzona analiza wykazała znaczącą przewagę planowania automatycznego nad ręcznym w zakresie czasu przygotowania harmonogramu, spójności wyników oraz minimalizacji błędów.

Najistotniejszą różnicą jest redukcja czasu planowania z około 2 godzin do około 1 minuty, co stanowi ponad stukrotne przyspieszenie procesu. Dodatkowo zastosowanie modelu optymalizacyjnego eliminuje konflikty zmian oraz ogranicza subiektywność decyzji, zwiększając sprawiedliwość i przewidywalność harmonogramu.

Uzyskane wyniki potwierdzają praktyczną użyteczność systemu w dynamicznym środowisku pracy gastronomii.

7. Podsumowanie i wnioski

7.1. Osiągnięte cele pracy

Celem niniejszej pracy inżynierskiej było zaprojektowanie oraz wdrożenie systemu informatycznego, który wspiera planowanie harmonogramów pracy w branży gastronomicznej. Zagadnienie przedstawione we wstępie pracy wynika z dużej zmienności warunków funkcjonowania gastronomii, w tym wahań zapotrzebowania na pracowników, a także z potrzeby uwzględniania wielu ograniczeń organizacyjnych i indywidualnych związanych z personelem.

W pracy przedstawiono projekt oraz implementację systemu przeznaczonego do zarządzania danymi pracowników, rejestrowania ich dyspozycyjności oraz tworzenia harmonogramów pracy. W ramach projektu wdrożono funkcje ręcznego układania grafików przez menedżera oraz moduł automatycznego planowania, którego działanie oparto na algorytmach optymalizacyjnych.

W pracy zastosowano bibliotekę OR-Tools oraz solver CP-SAT do zbudowania modelu optymalizacyjnego, w którym ujęto podstawowe ograniczenia, w tym dostępność pracowników oraz limity liczby godzin pracy. Opracowany moduł automatycznego planowania pełni funkcję narzędzia wspierającego decyzje administratora systemu, ponieważ pozwala w krótkim czasie generować harmonogramy zgodne z wcześniej określonymi warunkami.

Na podstawie przeprowadzonych testów oraz analizy działania systemu można stwierdzić, że cele pracy zostały osiągnięte. Zaproponowane rozwiązanie usprawnia proces tworzenia oraz aktualizowania harmonogramów pracy, co potwierdzają uzyskane wyniki i obserwacje z etapu weryfikacji.

7.2. Ograniczenia rozwiązania

Przedstawione rozwiązanie ma ograniczenia, które wynikają z przyjętych założeń projektowych oraz z uproszczeń zastosowanych w modelu optymalizacyjnym. System zaprojektowano z myślą o zaspokojeniu podstawowych potrzeb planowania grafików w gastronomii. W obecnym kształcie nie uwzględnia on pełnego zakresu złożoności przepisów prawa pracy ani rozbudowanych preferencji pracowników.

Przyjęty model opiera się na podstawowych ograniczeniach, takich jak dostępność zasobów oraz proste limity godzinowe. W tej postaci pomija on bardziej złożone reguły organizacyjne i operacyjne, które w praktyce mogą wpływać na poprawność oraz użyteczność otrzymanego harmonogramu. Zagadnienia te odnoszą się do rotacji zmian, poziomu kwalifikacji oraz zmieniających się priorytetów pracowników. Zastosowane uproszczenia wprowadzono celowo jako element decyzji projektowej, aby utrzymać przejrzystość modelu oraz zapewnić stabilną pracę solvera.

System nie uwzględnia też mechanizmów skalowania ani obsługi pracy w środowiskach, w których liczba użytkowników jest bardzo duża. Wskazane ograniczenia nie naruszają poprawności działania systemu w przyjętym zakresie oraz nie podważają rezultatów uzyskanych w ramach przeprowadzonych prac.

7.3. Możliwości dalszego rozwoju

Opracowany system pozwala na dalsze prace rozwojowe zarówno w obszarze funkcji, jak i stosowanych algorytmów. Jednym z kluczowych kierunków dalszych prac jest rozbudowa modelu optymalizacyjnego przez wprowadzenie dodatkowych ograniczeń. W szczególności warto uwzględnić bardziej złożone reguły dotyczące czasu pracy, tak aby model lepiej odzwierciedlał warunki organizacyjne i prawne.

Możliwe jest także rozszerzenie interfejsu użytkownika o bardziej zaawansowane mechanizmy wizualizacji harmonogramów oraz o narzędzia analityczne, które wspierają podejmowanie decyzji menedżerskich. System można rozbudować o mechanizmy automatycznego reagowania na zmiany w dostępności pracowników, w tym na nagłe nieobecności, tak aby ograniczyć opóźnienia w planowaniu i utrzymać ciągłość realizacji zadań.

Przyjęte w pracy podejście może stanowić punkt wyjścia do dalszych badań nad zastosowaniem algorytmów optymalizacyjnych w zarządzaniu zasobami ludzkimi. Jednocześnie wyniki mogą znaleźć zastosowanie w rzeczywistych systemach wspierających planowanie i organizację pracy personelu w branży gastronomicznej.

Bibliografia

1. LIPIEC, Stanisław. *Gastronomia. Praca i prekariat. Studium socjologiczno-prawne.* Wydawnictwo RYS, 2022. ISBN 9788367287104. Dostępne także z: https://books.google.pl/books?id=uEtzEAAAQBAJ.
2. IBM. *What is optimization modeling?* [https://www.ibm.com/think/topics/optimization-model]. 2023. Ostatni dostęp: 2 stycznia 2026.
3. BŁĄDEK, Iwo; MENSFELT, Agnieszka; MIAZGA, Konrad. *Autostopem przez gALAIktykę: Intuicyjne omówienie zagadnień – Tom I: Optymalizacja* [http://www.cs.put.poznan.pl/ibladek/students/ziwm/lab_pl/guide_opt.pdf]. [B.d.]. Ostatni dostęp: 2 stycznia 2026.
4. KRZESZOWSKA, Bogumiła; TRZASKALIK, Tadeusz; NOCOŃ, Ewa. Zastosowanie algorytmów genetycznych do układania planów lekcji na przykładzie Szkoły Podstawowej nr 5 w Olkuszu. *Prace Naukowe Uniwersytetu Ekonomicznego we Wrocławiu* [https://dbc.wroc.pl/Content/121726/Krzeszowska_Trzaskalik_Nocon_Zastosowanie_agorytmow_genetycznych.pdf]. 2010, nr. 108.
5. *Ustawa z dnia 26 czerwca 1974 r. – Kodeks pracy* [https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20250000277]. [B.d.]. Tekst jednolity, Ostatni dostęp: 2 stycznia 2026.
6. GOOGLE. *About OR-Tools* [https://developers.google.com/optimization/introduction]. 2024. Ostatni dostęp: 2 stycznia 2026.
7. GOOGLE. *Constraint Optimization* [https://developers.google.com/optimization/cp]. 2024. Ostatni dostęp: 2 stycznia 2026.
8. GOOGLE. *CP-SAT Solver* [https://developers.google.com/optimization/cp/cp_solver]. 2024. Ostatni dostęp: 2 stycznia 2026.
9. DA VINCI STUDIO. *Architektura aplikacji internetowej i proces tworzenia aplikacji webowych* [https://www.davinci-studio.com/pl/blog/architektura-aplikacji-internetowej/]. 2024. Ostatni dostęp: 2 stycznia 2026.
10. WIEGERS, K.; HOKANSON, C.; WÓJCICKI, Ł.; HELION. *Specyfikacja wymagań oprogramowania: kluczowe praktyki analizy biznesowej.* Helion, 2024. ISBN 9788328904361.
11. PINEDO, M. *Scheduling: Theory, Algorithms, and Systems.* Springer Science Business Media, 2012. Mathematics and Statistics. ISBN 9781461423614.

Spis rysunków

3.1 Przepływ danych w systemie planowania harmonogramu pracy 13

4.1 Diagram przypadków użycia systemu planowania harmonogramu pracy 16
4.2 Diagram aktywności procesu generowania harmonogramu pracy 17
4.3 Diagram aktywności procesu wprowadzania dyspozycyjności pracownika 18
4.4 Diagram aktywności procesu zarządzania pracownikami 19
4.5 Diagram aktywności procesu definiowania zmian roboczych 20
4.6 Diagram aktywności procesu zarządzania stanowiskami 21
4.7 Schemat ERD bazy danych systemu planowania harmonogramu pracy 26

5.1 Widok laptopowy administratora: planer zmian 35
5.2 Widok laptopowy administratora: ustawienia (użytkownicy i stanowiska) 36
5.3 Widok laptopowy administratora: przygotowanie danych do solvera 37
5.4 Widok laptopowy administratora: projektowanie zmian i reguł 38
5.5 Widoki raportowe 39
5.6 Widoki mobilne użytkownika 40

Spis tabel

2.1 Zestawienie zmiennych decyzyjnych modelu 9
2.2 Podział ograniczeń w modelu planowania grafiku pracy 10

4.1 Główne encje bazy danych systemu planowania harmonogramu pracy 23
4.2 Typy relacji pomiędzy głównymi encjami bazy danych 24
4.3 Hasłowe powiązanie funkcjonalności systemu z encjami bazy danych 25

5.1 Zestawienie głównych endpointów zaimplementowanych w systemie 28

6.1 Powiązanie wymagań funkcjonalnych ze scenariuszami testowymi 44
6.2 Czas działania solvera dla różnych rozmiarów danych 45
6.3 Porównanie planowania ręcznego i automatycznego 46

Spis kodów źródłowych

5.1 Ograniczenie pojemności z uwzględnieniem przypisań istniejących 29
5.2 Blokada duplikatów względem przypisań istniejących 30
5.3 Ograniczenie kompetencyjne stanowisk 30
5.4 Blokada przypisań kolidujących z dyspozycyjnością i istniejącym grafikiem 30
5.5 Zakaz nakładania nowych przypisań tego samego użytkownika 31
5.6 Konstrukcja nieobsadzonych miejsc i celu fazy 1 32
5.7 Minimalizacja przydziałów dla wagi 0 przy zachowaniu pokrycia 32
5.8 Warunkowa minimalizacja globalnego spreadu obciążenia 32
5.9 Minimalizacja rozpiętości relacji minutes/target 33
5.10 Priorytet dla użytkowników z explicit target 33
5.11 Funkcja fairness (balans godzin) 33
5.12 Końcowa funkcja jakości z utrzymaniem fairness 34